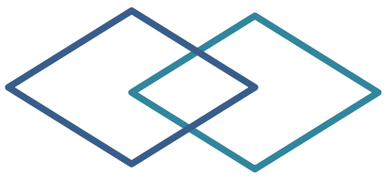


第5章 通用输入输出GPIO模块



本章知识与能力要求

- 了解GPIO的基本概念；
- 理解STM32F103微控制器GPIO的内部结构、工作模式和使用特性；
- 理解GPIO的输入输出模式；
- 熟悉STM32F103微控制器GPIO相关的标准外设库函数和HAL库函数；
- 掌握基于标准外设库实现LED灯闪烁的方法；
- 掌握基于HAL库实现LED灯闪烁的方法。

第5章 通用输入输出GPIO模块


5.1 GPIO概述

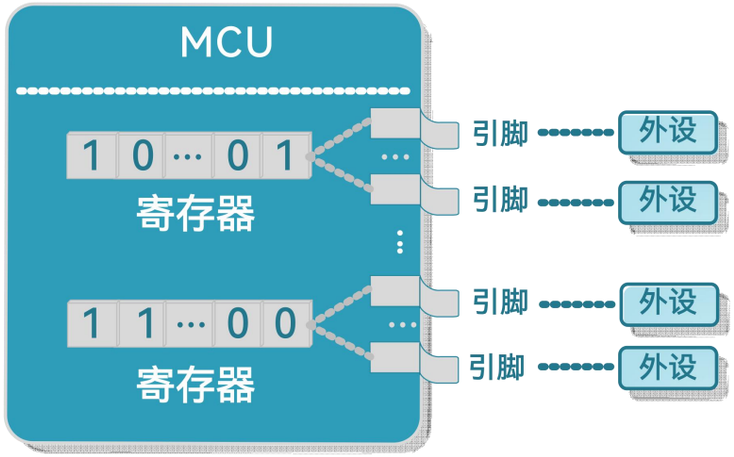

5.2 STM32的GPIO工作原理


5.3 GPIO模块的标准外设库接口函数及应用


5.4 GPIO模块的HAL库接口函数及应用

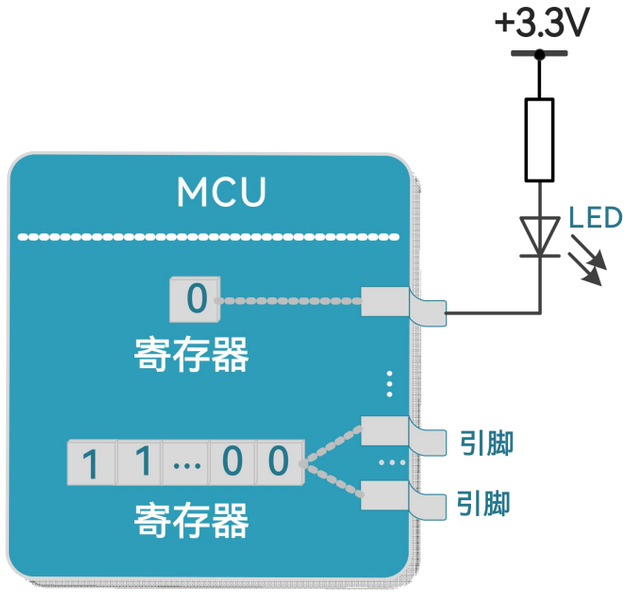

5.5 编程思想之模块化编程

5.1 GPIO概述

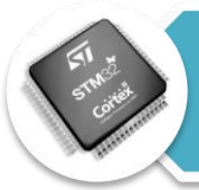


GPIO端口内部是由一个个寄存器组成，一个引脚对应一个寄存器或多个引脚对应一个多位的寄存器，改变寄存器中的数据就可以改变外设的工作方式。

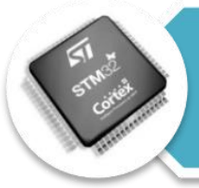
与51单片机相比，STM32拥有更多的I/O引脚，其驱动能力更强，控制方式更多更灵活，功能也更强大。



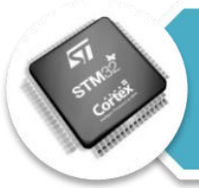
5.2 STM32的GPIO工作原理



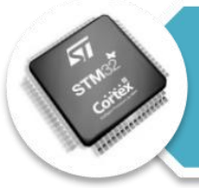
5.2.1 GPIO引脚



5.2.2 GPIO内部结构

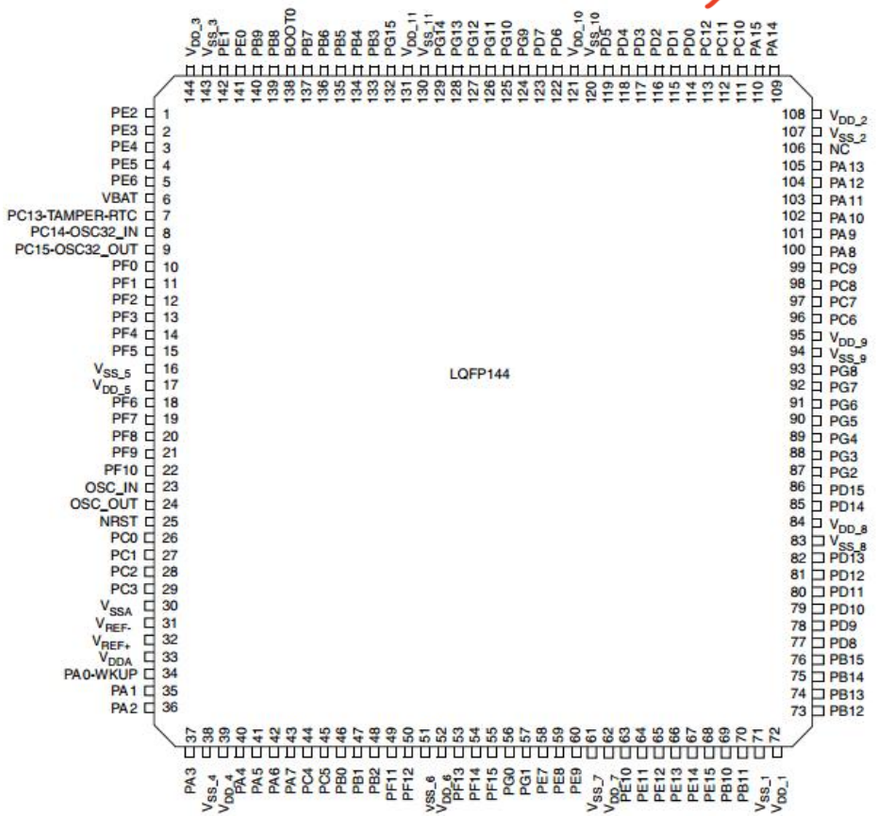


5.2.3 GPIO工作模式



5.2.4 GPIO输出速度

5.2.1 STM32F103引脚图



STM32F103ZET6一共有144个引脚，这些引脚共分为六大类：

& 电源引脚：V_{DD}、V_{SS}、V_{REF+}、V_{REF-}、V_{DDA}、V_{SSA}、、V_{BAT}；

& 晶振引脚：PC14、PC15和OSC_IN、OSC_OUT

& 复位引脚：NRST。

& BOOT引脚：BOOT0、BOOT1；

& 程序下载引脚：PA13、PA14、PA15、PB3和PB4

& GPIO引脚：共有7组GPIO，分别为：

PA组：PA0~PA15； PB组：PB0~ PB15；

PC组：PC0~ PC15； PD组：PD0~ PD15；

PE组：PE0~PE15； PF组：PF0~PF15；

PG组：PG0~PG15；

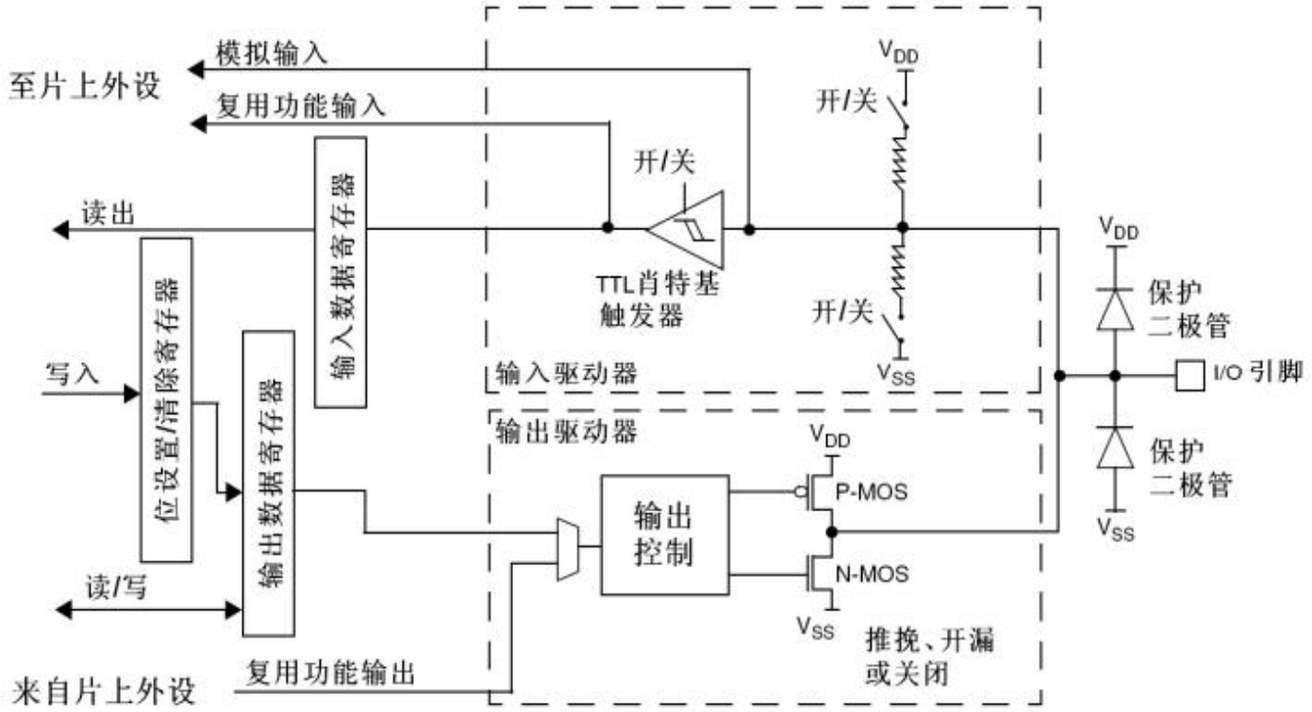
大多数引脚还通过复用技术兼具其他专用功能。

5.2.2 GPIO内部结构

左侧连接MCU内部

中间部分为GPIO引脚的基本电路。

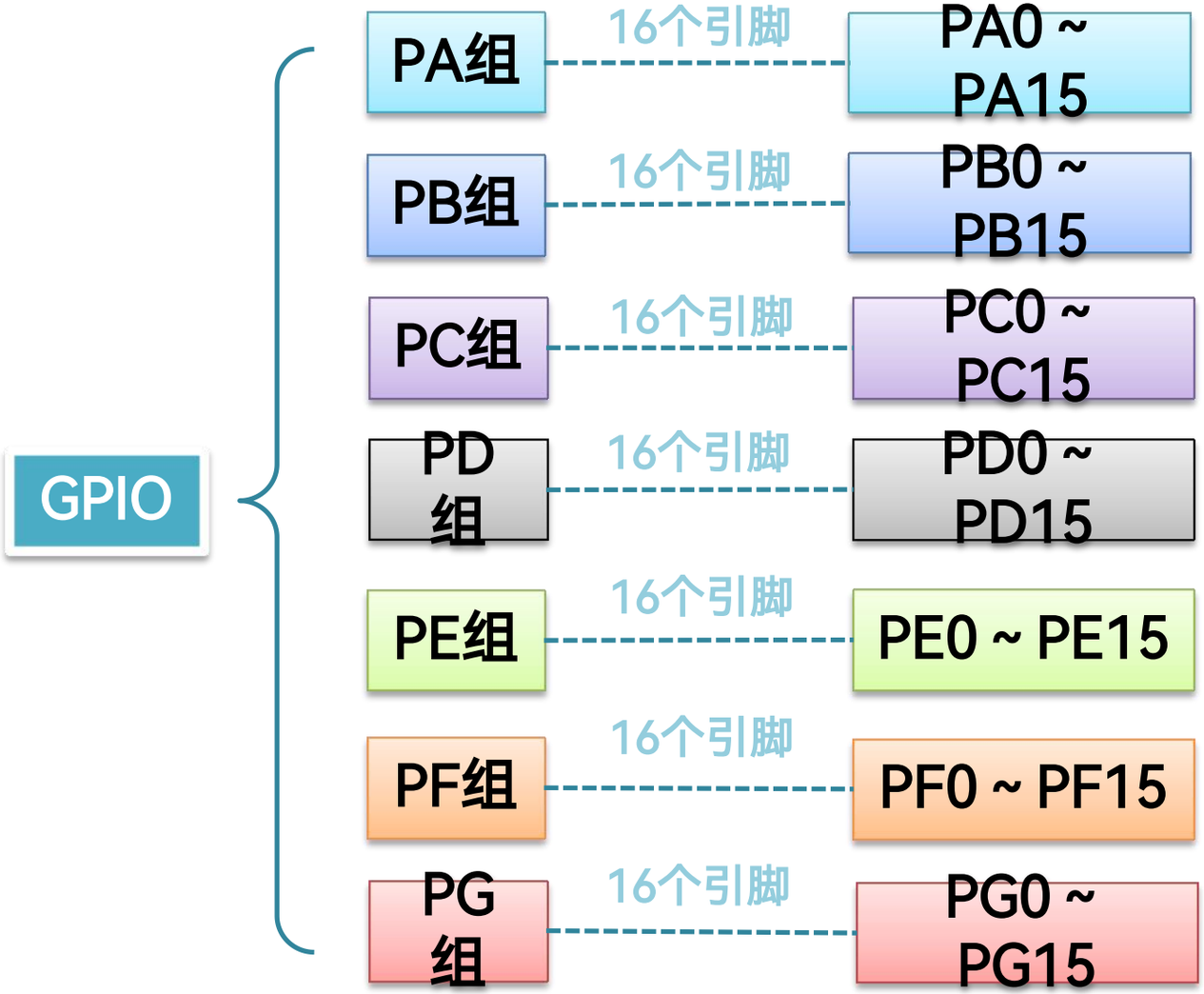
右侧为MCU引出的I/O外设引脚



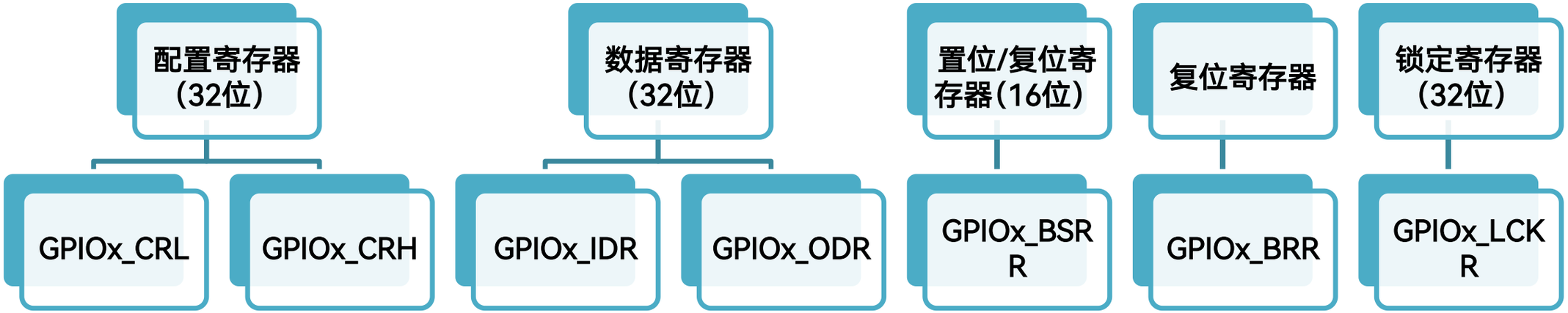
STM32F103系列微控制器GPIO端口的基本结构

5.2.2 GPIO内部结构

STM32F103系列微控制器共有7组GPIO端口，每一组端口包括16个引脚，如PA组端口，包括PA0~PA15共16个引脚。

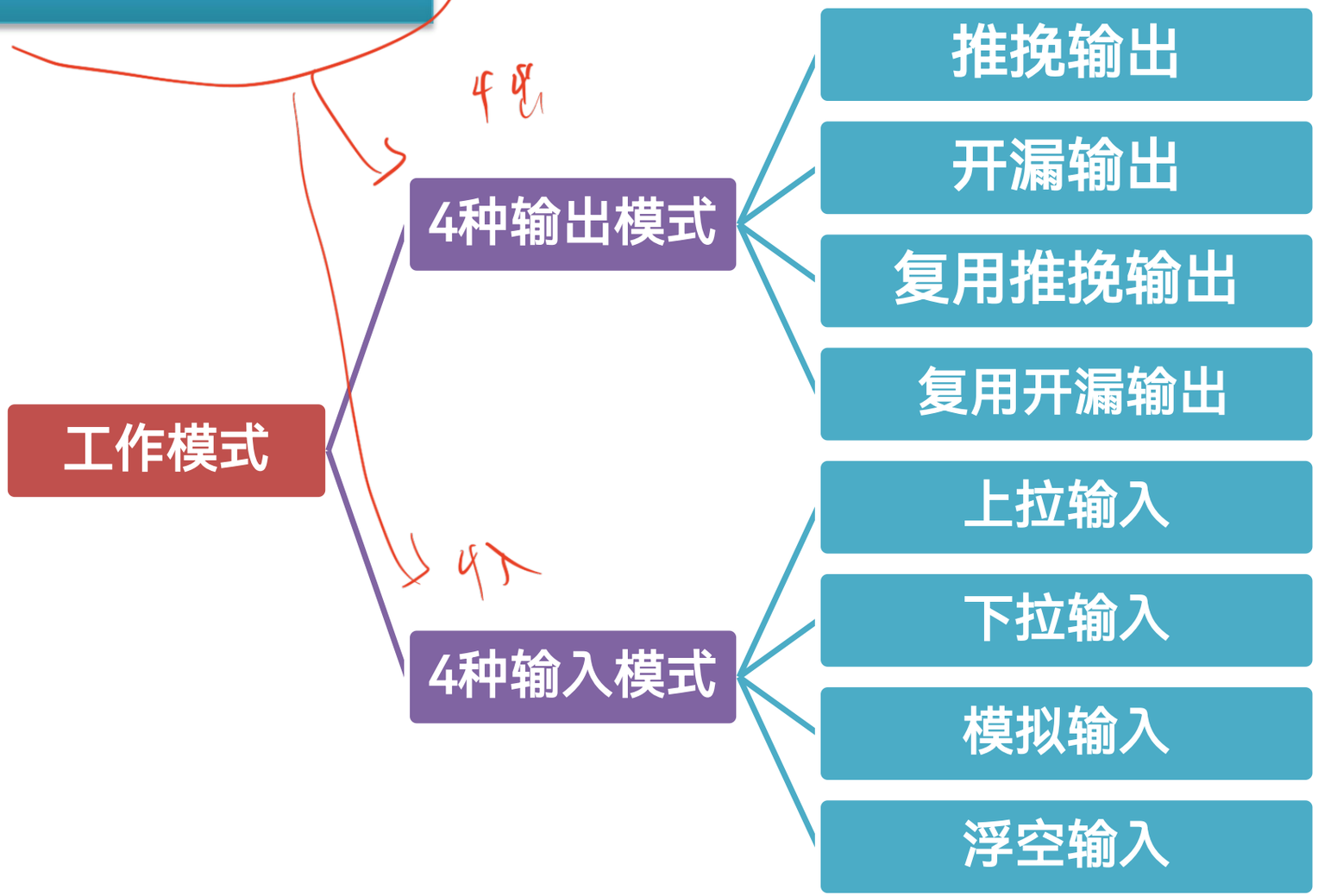


5.2.2 GPIO内部结构

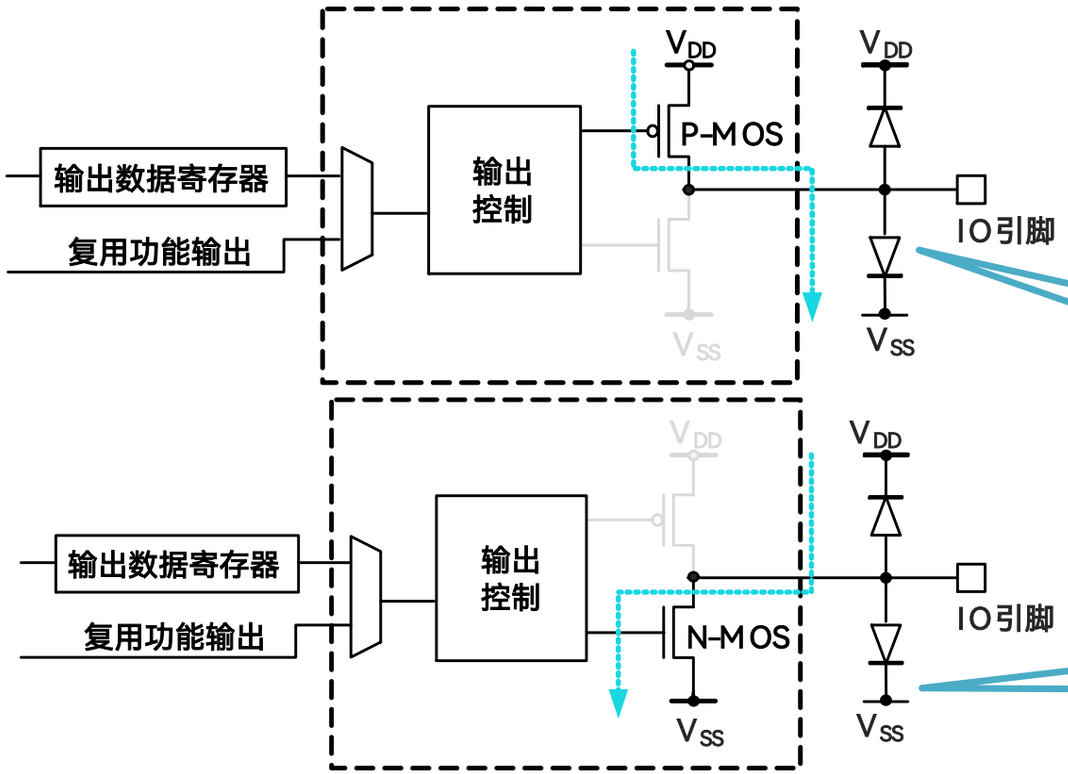


每组GPIO端口(Px)都由7个寄存器组成，负责控制该端口的16个引脚Px0 ~ Px15。

5.2.3 GPIO工作模式



推挽输出模式(Push-Pull，PP)



推挽结构一般指两个MOS管受互补信号的控制，按互补对称的方式连接，任意时刻总是一个三极管导通，另一个截止。

推挽模式下，I/O引脚输出高电平时，P-MOS导通

推挽模式下，I/O引脚输出低电平时，N-MOS导通

使用推挽输出模式的目的：增大输出电流，即增加输出引脚的驱动能力，提高电路的负载能力和提高开关的速度。

开漏输出模式(Open-Drain，OD)

漏极开路(OD）输出与集电极开路输出十分相似，即只有下拉MOS管没有上拉MOS管，MOS管的漏极直接与I/O引脚相连，不与电源连接，处于悬空状态，称之为漏极开路。



开漏输出模式的应用场合

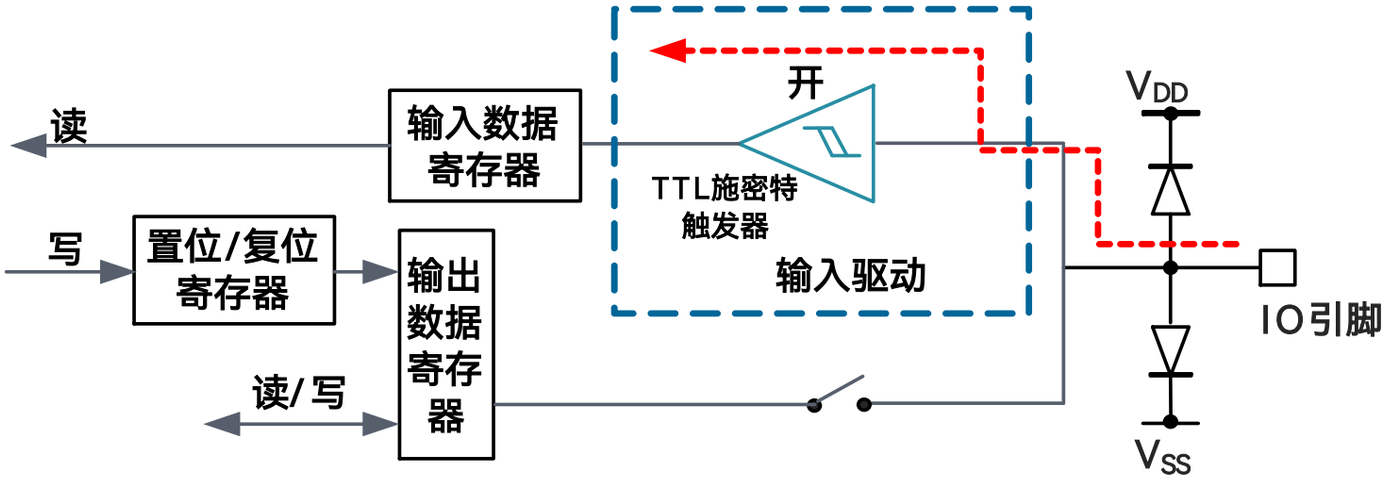
复用功能输出模式(Alternate Function，AF)

STM32F103系列微控制器有144个引脚，其中的GPIO引脚除了作为通用输入/输出引脚使用外，还可以作为片上外设(如串口、ADC、IIC等）的I/O引脚，即一个引脚可以作为多个外设引脚使用，称为复用I/O端口AFIO(Alternate Function I/O），但一个引脚某一时刻只能使用复用功能中的一个。

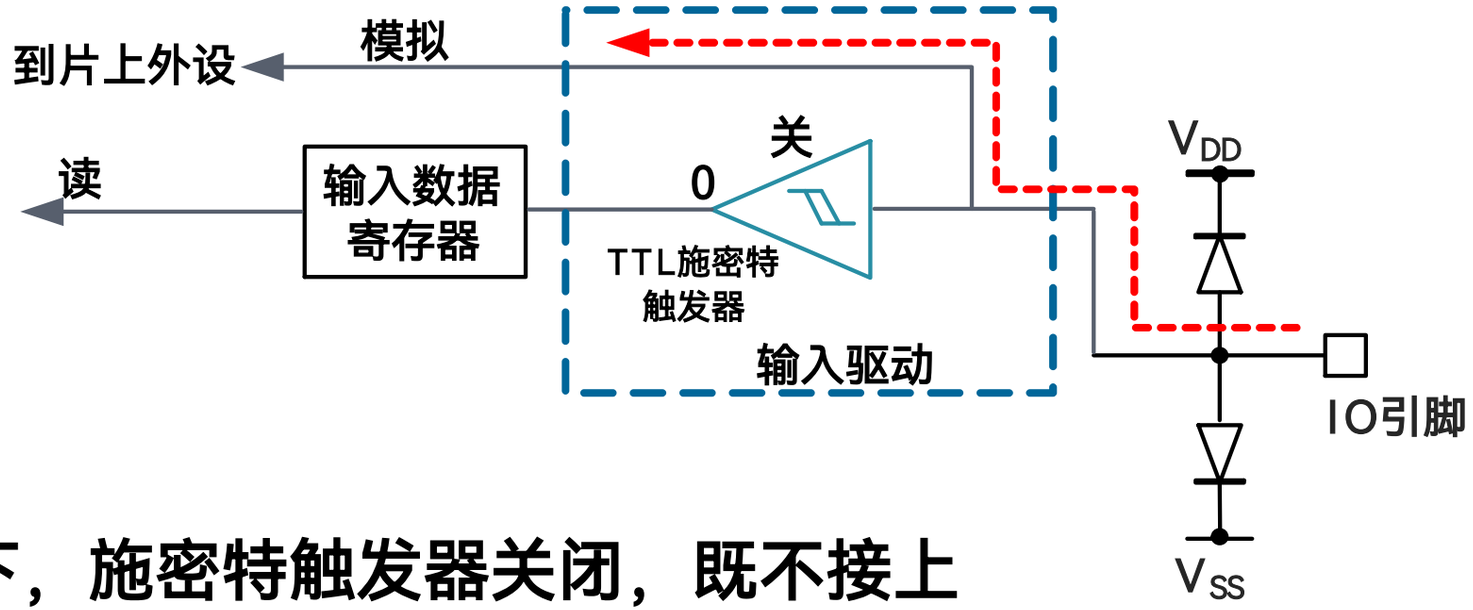
当I/O引脚作为复用功能时，可选择复用推挽输出模式或复用开漏输出模式，在选择复用开漏输出模式时，需外接上拉电阻。

输入模式

上拉输入模式(Input Pull-up)	上拉输入模式，引脚内部有个上拉电阻，通过开关连接到电源VDD，当I/O引脚无输入信号时，默认输入高电平。
下拉输入模式(Input Pull-down)	下拉输入模式和上拉输入模式正好相反，当I/O引脚无输入信号时，默认输入低电平。
浮空输入模式	浮空输入模式下引脚内部既不接上拉电阻也不连接下拉电阻，直接经施密特触发器输入I/O引脚的信号。



模拟输入模式



模拟输入模式下，施密特触发器关闭，既不接上拉电阻也不连接下拉电阻，引脚信号连接到芯片内部的片上外设，其典型应用是A/D模拟输入，对外部信号进行采集。

5.2.4 GPIO输出速度

输出速度并不是输出信号的速度，而是指I/O口驱动电路的响应速度。

STM32F103系列微控制器的I/O引脚的输出速度有3种选择：2MHz、10MHz和50MHz。

实际开发中，需要结合系统实际情况选择合适的响应速度，以确保信号的稳定性和降低功耗等。

一般常用的外设如LED、蜂鸣器等建议采用2MHz的输出速度，而用作IIC、SPI等复用功能的输出引脚时，则尽量选择高响应速度如10MHz或50MHz。

5.3 GPIO模块的标准外设库接口函数及应用



5.3.1 GPIO标准外设库接口函数



5.3.2 GPIO标准外设库应用实例



5.3.3 基于标准外设库开发的一般流程

5.3.1 GPIO标准外设库接口函数

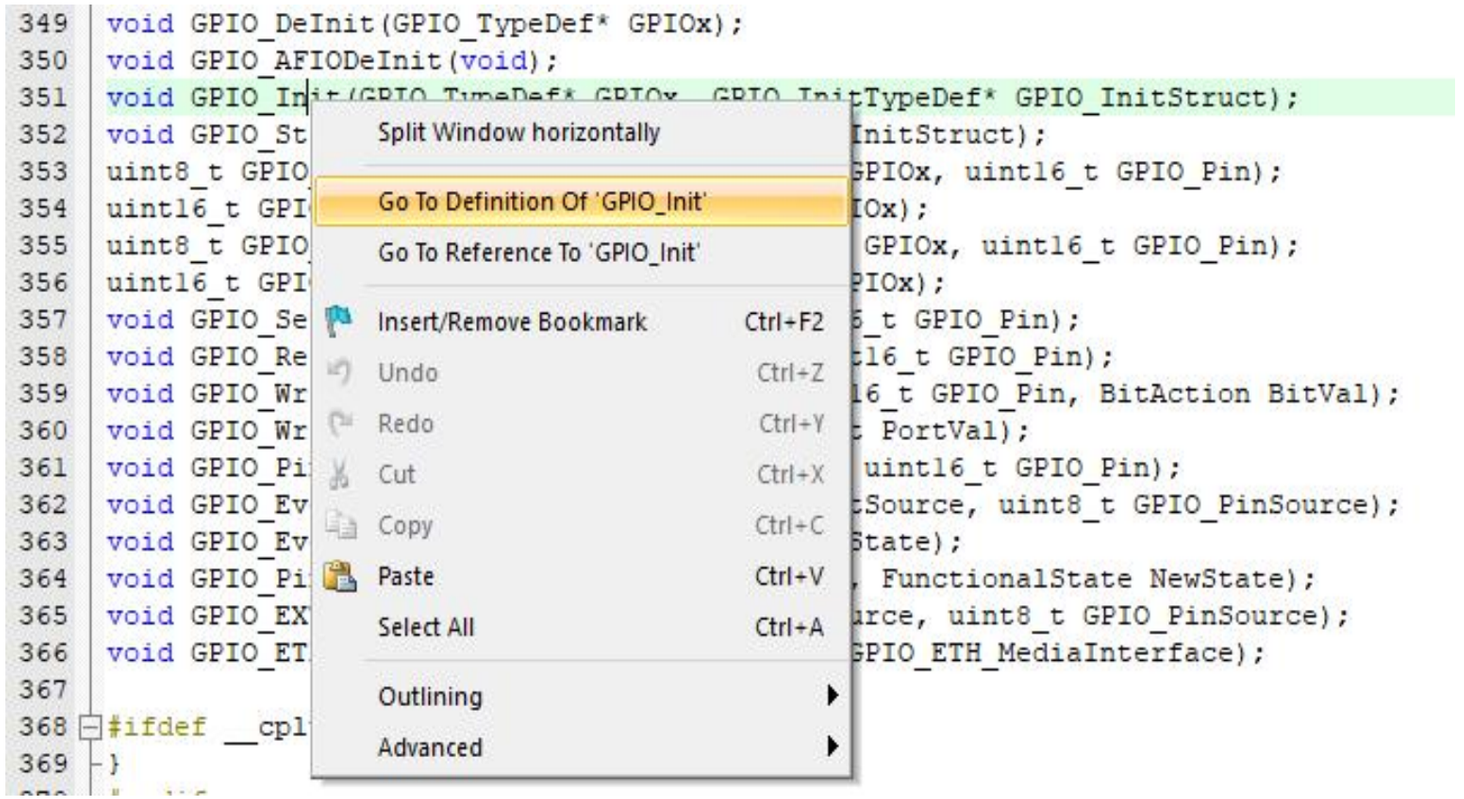
类型	函数原型	功能描述
初始化及复位函数	GPIO_Init()	根据GPIO初始化结构参数初始化GPIOx外设寄存器
	GPIO_DeInit()	将GPIOx外设寄存器恢复为默认复位值
	GPIO_StructInit()	GPIO结构体初始化
	GPIO_AFIODeInit()	取消所有的复用功能
引脚功能操作函数	GPIO_Write()	置位或复位选定的GPIO数据端口
	GPIO_WriteBit()	置位或复位选定的GPIO端口引脚
	GPIO_SetBits()	置位选定的GPIO端口引脚
	GPIO_ResetBits()	复位选定的GPIO端口引脚
	GPIO_ReadInputData()	读取选定的GPIO输入端口数据
	GPIO_ReadInputDataBit()	读取选定的GPIO输入端口引脚数据
	GPIO_ReadOutputDataBit()	读取选定的GPIO输出端口数据
	GPIO_ReadOutputData()	读取选定的GPIO输出端口引脚数据
外部中断处理函数	GPIO_EXTILineConfig();	外设端口作为中断线号输入
	GPIO_EventOutputConfig()	事件输出

5.3.1 GPIO标准外设库接口函数

GPIO的标准外设库接口函数的源码在源文件stm32f10x_gpio.c中，其对应的头文件stm32f10x_gpio.h声明了GPIO所有的库函数，共18种。

```
345 /** @defgroup GPIO_Exported_Functions
346   * @{
347   */
348
349 void GPIO_DeInit(GPIO_TypeDef* GPIOx);
350 void GPIO_AFIODeInit(void);
351 void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct);
352 void GPIO_StructInit(GPIO_InitTypeDef* GPIO_InitStruct);
353 uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
354 uint16_t GPIO_ReadInputData(GPIO_TypeDef* GPIOx);
355 uint8_t GPIO_ReadOutputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
356 uint16_t GPIO_ReadOutputData(GPIO_TypeDef* GPIOx);
357 void GPIO_SetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
358 void GPIO_ResetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
359 void GPIO_WriteBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, BitAction BitVal);
360 void GPIO_Write(GPIO_TypeDef* GPIOx, uint16_t PortVal);
361 void GPIO_PinLockConfig(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
362 void GPIO_EventOutputConfig(uint8_t GPIO_PortSource, uint8_t GPIO_PinSource);
363 void GPIO_EventOutputCmd(FunctionalState NewState);
364 void GPIO_PinRemapConfig(uint32_t GPIO_Remap, FunctionalState NewState);
365 void GPIO_EXTILineConfig(uint8_t GPIO_PortSource, uint8_t GPIO_PinSource);
366 void GPIO_ETH_MediaInterfaceConfig(uint32_t GPIO_ETH_MediaInterface);
367
```

5.3.1 GPIO标准外设库接口函数



如果想要查看具体的函数定义实现，可在keil 5工程中将光标放在想要查看的函数上，然后右键选择“Go To Definition of…”，即可跳转到相应函数的函数体中

5.3.1 GPIO标准外设库接口函数

```
/**
  * @brief  Initializes the GPIOx peripheral according to the specified
  *         parameters in the GPIO_InitStruct.
  * @param  GPIOx: where x can be (A..G) to select the GPIO peripheral.
  * @param  GPIO_InitStruct: pointer to a GPIO_InitTypeDef structure that
  *         contains the configuration information for the specified GPIO peripheral.
  * @retval None
  */
void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct)
{
  uint32_t currentmode = 0x00, currentpin = 0x00, pinpos = 0x00, pos = 0x00;
  uint32_t tmpreg = 0x00, pinmask = 0x00;
  /* Check the parameters */
  assert_param(IS_GPIO_ALL_PERIPH(GPIOx));
  assert_param(IS_GPIO_MODE(GPIO_InitStruct->GPIO_Mode));
  assert_param(IS_GPIO_PIN(GPIO_InitStruct->GPIO_Pin));

/*---------------------------- GPIO Mode Configuration -----------------------*/
  currentmode = ((uint32_t)GPIO_InitStruct->GPIO_Mode) & ((uint32_t)0x0F);
  if ((((uint32_t)GPIO_InitStruct->GPIO_Mode) & ((uint32_t)0x10)) != 0x00)
  {
    /* Check the parameters */
    assert_param(IS_GPIO_SPEED(GPIO_InitStruct->GPIO_Speed));
    /* Output mode */
    currentmode |= (uint32_t)GPIO_InitStruct->GPIO_Speed;
  }
/*---------------------------- GPIO CRL Configuration ------------------------*/
  /* Configure the eight low port pins */
  if (((uint32_t)GPIO_InitStruct->GPIO_Pin & ((uint32_t)0x00FF)) != 0x00)
```

GPIO_Init()函数源码

GPIO_Init()函数用于初始化GPIO，有两个输入参数：

- 第一个参数用于指定GPIO的具体端口x（x=A,B,C,D,E）
- 第二个参数根据GPIO_InitStruct结构体变量中指定的参数初始化GPIO，GPIO_InitStruct是指向GPIO_InitTypeDef结构体的指针。

5.3.1 GPIO标准外设库接口函数

```
87  /**
88    * @brief  GPIO Init structure definition
89    */
90
91  typedef struct
92  {
93    uint16_t GPIO_Pin;              /*!< Specifies the GPIO pins to be configured.
94                                         This parameter can be any value of @ref GPIO_pins_define */
95
96    GPIOSpeed_TypeDef GPIO_Speed;   /*!< Specifies the speed for the selected pins.
97                                         This parameter can be a value of @ref GPIOSpeed_TypeDef */
98
99    GPIOMode_TypeDef GPIO_Mode;     /*!< Specifies the operating mode for the selected pins.
100                                        This parameter can be a value of @ref GPIOMode_TypeDef */
101 }GPIO_InitTypeDef;
```

GPIO_InitTypeDef结构体定义在stm32f10x_gpio.h

GPIO的配置参数，如具体的GPIO引脚、输出速度、GPIO引脚的工作模式等。

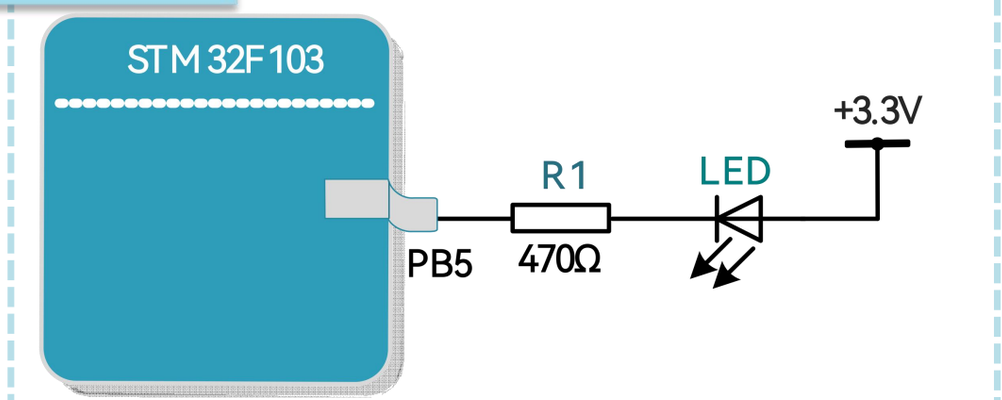
GPIO_InitTypeDef结构体成员	取值范围
GPIO_Pin	GPIO_Pin_0 ~ GPIO_Pin_15 GPIO_Pin_All
GPIO_Speed	GPIO_Speed_10MHz GPIO_Speed_2MHz GPIO_Speed_50MHz
GPIO_Mode	GPIO_Mode_AIN GPIO_Mode_IN_FLOATING GPIO_Mode_IPD GPIO_Mode_IPU GPIO_Mode_Out_OD GPIO_Mode_Out_PP GPIO_Mode_AF_OD GPIO_Mode_AF_PP

5.3.2 GPIO标准外设库应用实例

功能

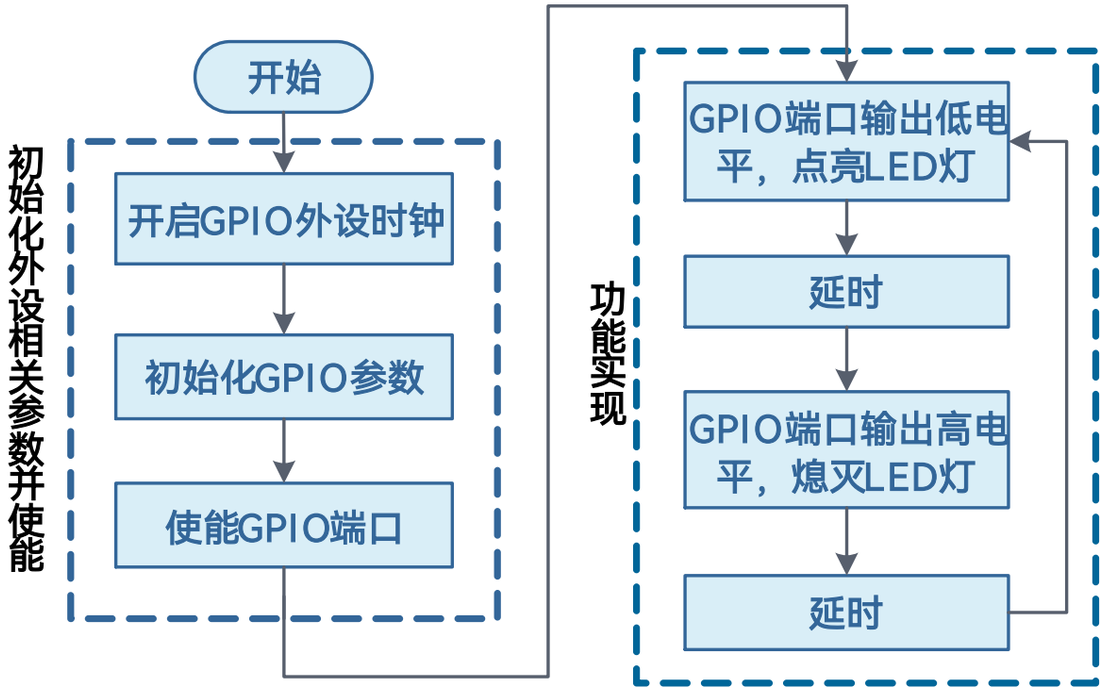
采用基于标准外设库设计方式，利用单个GPIO引脚输出高低电平控制发光二极管，并按一定时间间隔改变I/O口电平，实现灯光闪烁效果。

硬件设计



改变限流电阻R1的值可改变发光二极管的亮度，一般选用400 ~ 1K 电阻。

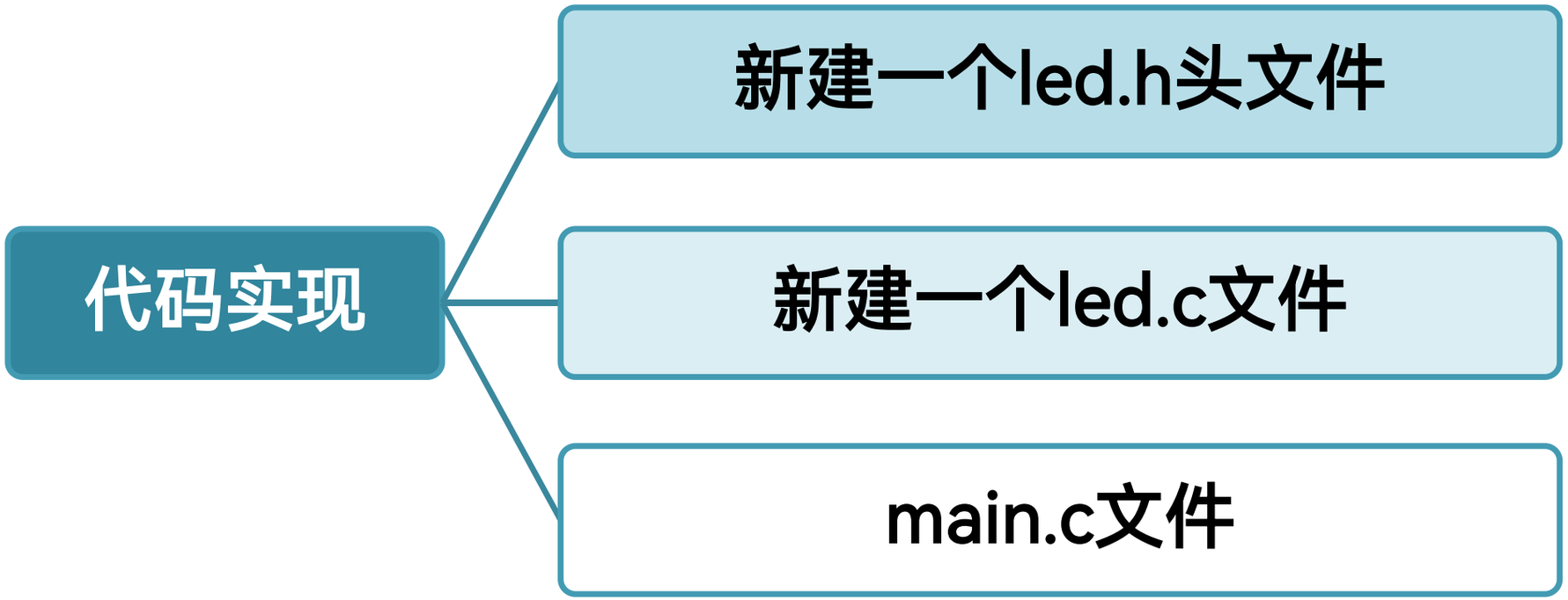
5.3.2 GPIO标准外设库应用实例



主流程图由两部分构成：

- 初始化函数部分；
- 一个无限循环功能部分。

5.3.2 GPIO标准外设库应用实例



5.3.2 GPIO标准外设库应用实例

led.h头文件

用于存放led.c文件的管脚定义、全局变量声明和函数声明等内容。

```
#ifndef __LED_H
#define __LED_H

#include "stm32f10x.h"
void LED_Init(void); //函数声明

#endif
```

5.3.2 GPIO标准外设库应用实例

新建一个led.c文件：

用于GPIO端口初始化操作，即硬件驱动程序的编写。

```
#include "led.h"
void LED_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
            //定义一个GPIO_InitTypeDef类型的结构体变量

RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB,ENABLE);
            //开启GPIOB的时钟
    PIO_InitStructure.GPIO_Pin = GPIO_Pin_5;
            //选择要使用的I/O引脚，此处选择PB5引脚
   GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
            //设置引脚输出模式为推挽输出
   GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
            //设置引脚的输出速度为50MHz
    GPIO_Init(GPIOB,&GPIO_InitStructure);
            //调用初始化库函数初始化GPIOB端口
```

5.3.2 GPIO标准外设库应用实例

main.c文件的代码

```
#include "stm32f10x.h"
#include "led.h"
/*函数名：Delay
*功能描述：不精确的延时，延时时间=nCount/72000，单位为ms，72MHz为STM32主频
*输入参数：nCount
*输出参数：无
*/
void Delay(__IO u32 nCount)
{
    for(;nCount !=0;nCount--);
}
```

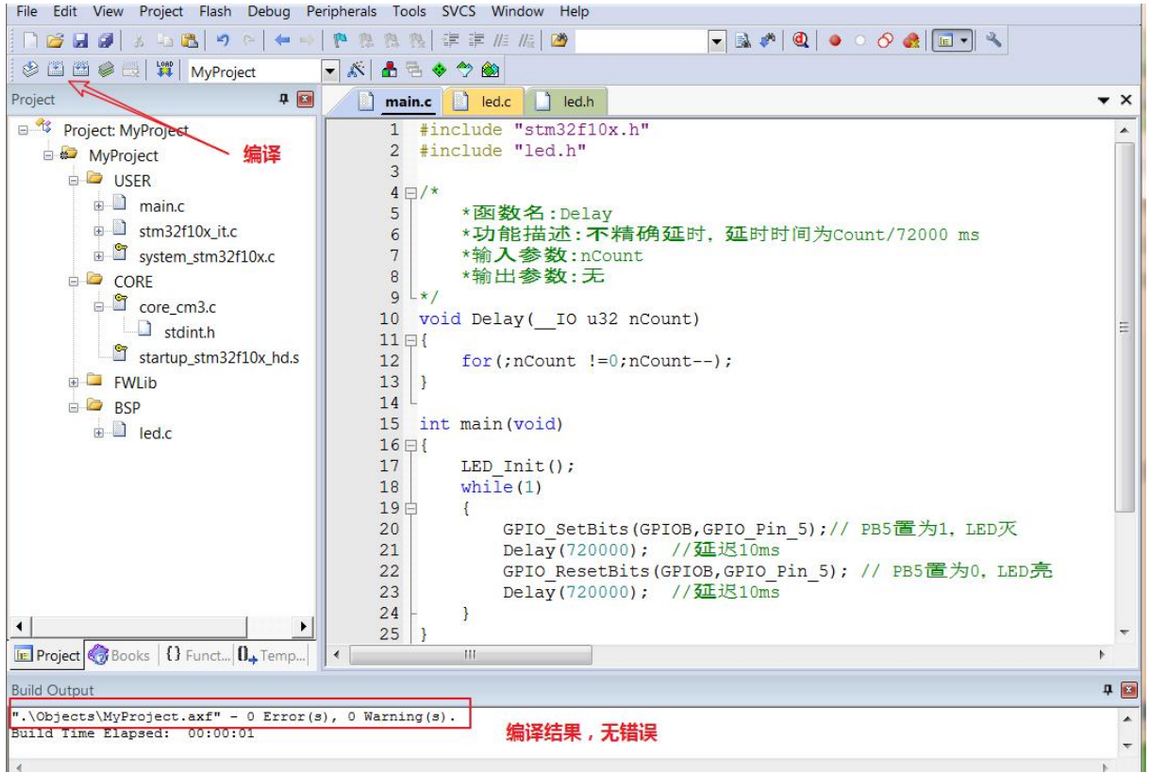
```
int main(void)
{
    LED_Init();
    while(1)
    {   GPIO_SetBits(GPIOB,GPIO_Pin_5);
        //调用GPIO_SetBits函数，将PB5置为高电平，熄灭LED
        Delay(720000);
        //调用延迟函数，延迟10ms
        GPIO_ResetBits(GPIOB,GPIO_Pin_5);
        //调用GPIO_ReSetBits函数，将PB5置为低电平，点亮LED
        Delay(720000);
        //调用延迟函数，延迟10ms
    }
}
```

5.3.2 GPIO标准外设库应用实例

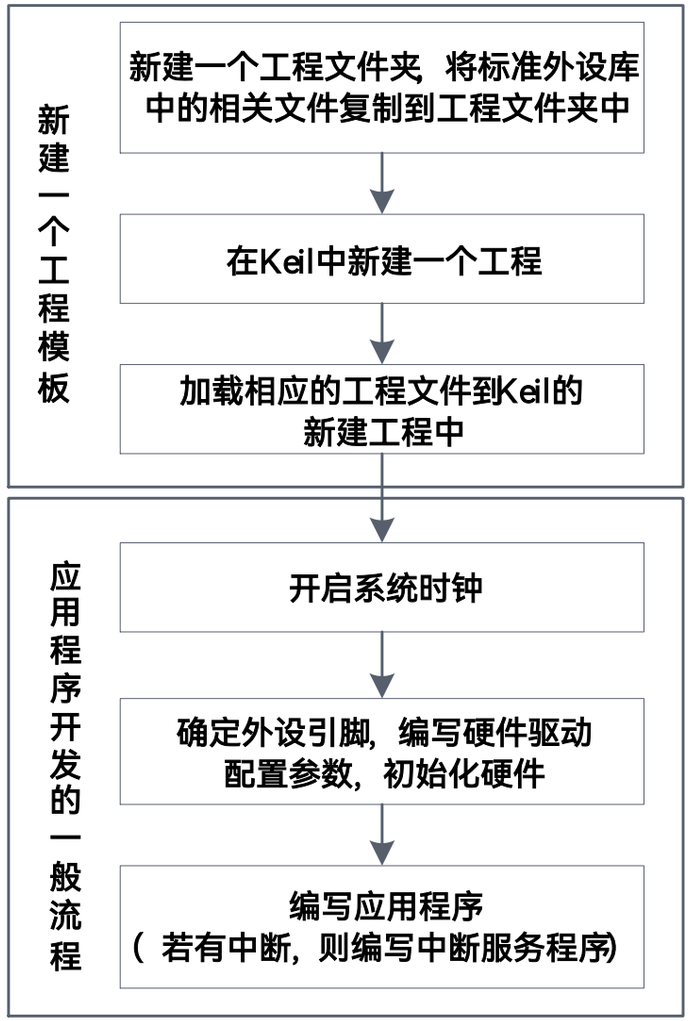
代码完成后，单击Keil中的编译按钮 进行编译；

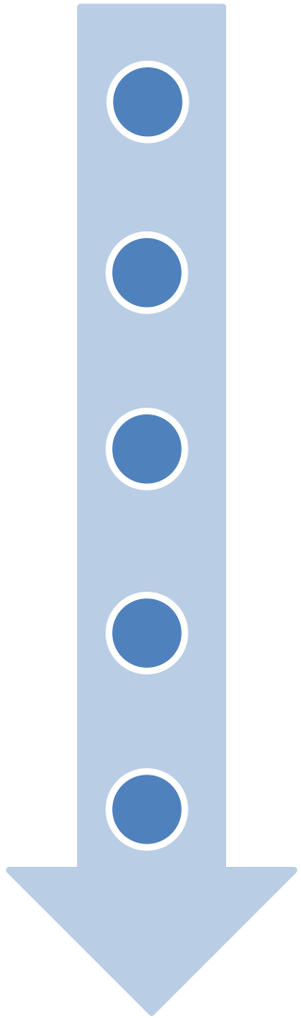
若提示程序有错误，则需认真检查一下代码，找出错误所在，直到编译无错误；

如果提示无错误，就可以通过调试和下载工具，将编译好的可执行文件下载到芯片中，查看并验证实验结果。



5.3.3 基于标准外设库开发的一般流程





开启相应外设时钟

确定功能引脚

初始化外设硬件

应用功能实现

若有中断，编写中断服务函数

- 开启相应外设时钟
- 配置外设功能参数，调用初始化函数，初始化外设相关的参数
- 使能相应的外设

5.4 GPIO模块的HAL库接口函数及应用



5.4.1 GPIO的HAL库接口函数

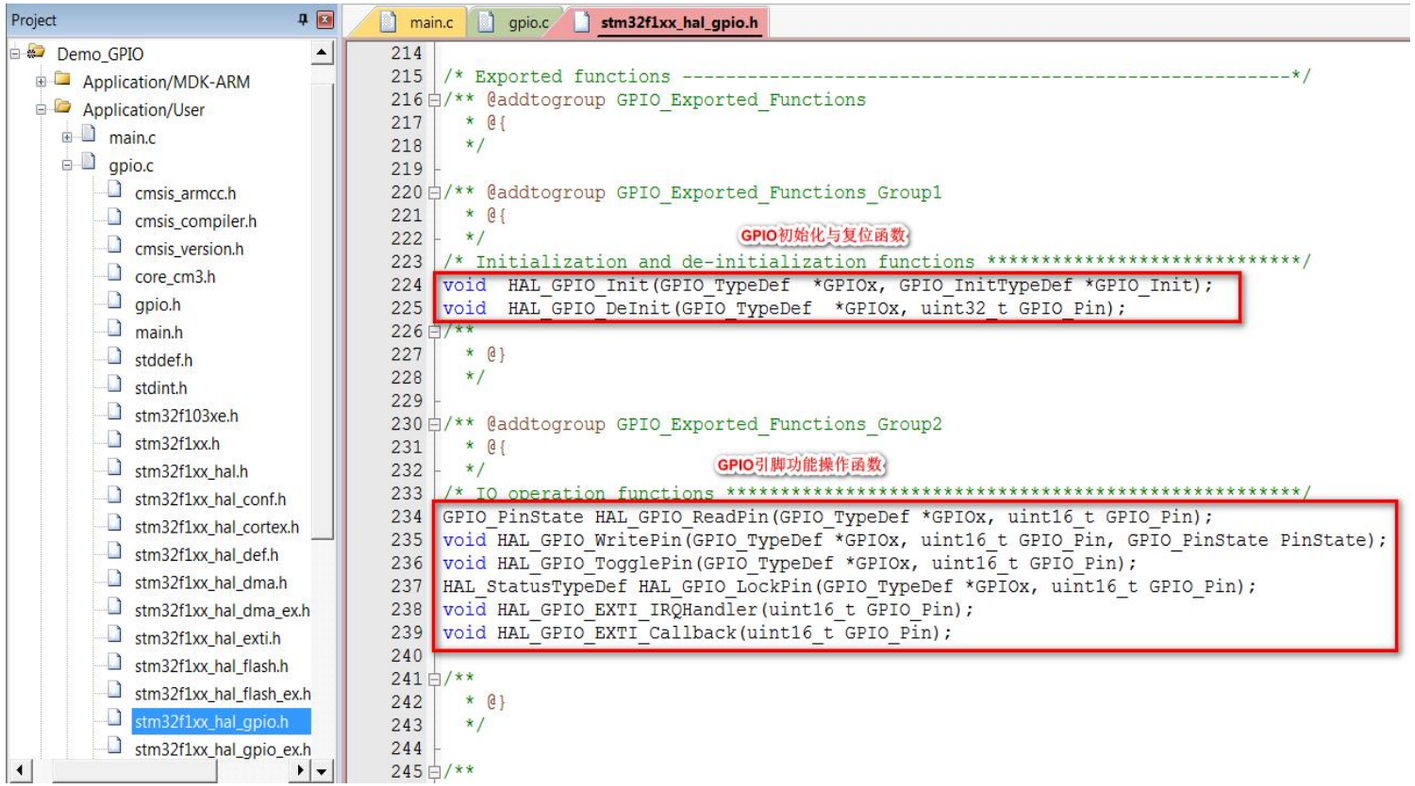


5.4.2 GPIO的 HAL库应用实例



5.4.3 基于HAL库开发的一般流程

5.4.1 GPIO的HAL库接口函数



GPIO的HAL库接口函数的源码在源文件stm32f1xx_hal_gpio.c中，其对应的头文件stm32f1xx_hal_gpio.h声明了GPIO所有的库函数，共8种。

5.4.1 GPIO的HAL库接口函数

类型	函数原型	功能描述
初始化及复位函数	HAL_GPIO_Init()	GPIO初始化函数
	HAL_GPIO_DeInit()	复位选定的端口引脚到初始状态
引脚功能操作函数	HAL_GPIO_ReadPin()	读取选定的端口引脚的电平状态
	HAL_GPIO_WritePin()	设置选定的端口引脚输出高电平或低电平
	HAL_GPIO_TogglePin()	设置选定端口引脚的电平状态翻转
	HAL_GPIO_LockPin()	当端口引脚电平状态改变时保持锁定时的值
	HAL_GPIO_EXTI_IRQHandler()	外部中断处理函数
	HAL_GPIO_EXTI_Callback()	中断回调函数

5.4.1 GPIO的HAL库接口函数

引脚功能操作函数

源码解析：

读取输入数据寄存器IDR的值与指定引脚进行按位与操作，结果若不为0，则返回高电平；结果若为0，则返回低电平。

GPIOx为引脚的端口号，取值为GPIOA~GPIOG，表示指向GPIO结构体的指针，用于访问端口引脚的寄存器。

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef
*GPIOx, uint16_t GPIO_Pin)
{
  GPIO_PinState bitstatus;

  if ((GPIOx->IDR & GPIO_Pin) !=
(uint32_t)GPIO_PIN_RESET)
  {
       bitstatus = GPIO_PIN_SET;
  }
  else
  {
       bitstatus = GPIO_PIN_RESET;
  }
  return bitstatus;
}
```

5.4.1 GPIO的HAL库接口函数

GPIO_Pin为常量

GPIO_Pin_0表示0x0001；

GPIO_Pin_15表示0x8000。

GPIO_PinState表示引脚电平状态，为枚举变量，

```
typedef enum
{
  GPIO_PIN_RESET = 0u,
  GPIO_PIN_SET
} GPIO_PinState;
```

取值范围为GPIO_PIN_SET =1或者GPIO_PIN_RESET = 0。

引脚号	引脚定义寄存器地址
GPIO_PIN_0	((uint16_t) 0x0001
GPIO_PIN_1	(uint16_t) 0x0002
GPIO_PIN_2	0x0004
GPIO_PIN_3	0x0008
GPIO_PIN_4	0x0010
GPIO_PIN_5	0x0020
GPIO_PIN_6	0x0040
GPIO_PIN_7	0x0080
GPIO_PIN_8	0x0100
GPIO_PIN_9	0x0200
GPIO_PIN_10	0x0400
GPIO_PIN_11	0x080
GPIO_PIN_12	0x1000
GPIO_PIN_13	0x2000
GPIO_PIN_14	0x4000
GPIO_PIN_15	0x8000
GPIO_PIN_All	0xFFFF

5.4.1 GPIO的HAL库接口函数

函数源码

```
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)
{
  if (PinState != GPIO_PIN_RESET)
  {
    GPIOx->BSRR = GPIO_Pin;
  }
  else
  {
    GPIOx->BSRR = (uint32_t)GPIO_Pin << 16U;
  }
}
```

源码解析

将指定引脚的电平状态写入置位/复位寄存器BSRR中，该寄存器为32位，其中高16位控制端口16个引脚(0~15）输出低电平，低16位控制端口16个引脚输出高电平。若设置的引脚电平状态不等于低电平，则将该引脚设置的状态位写入BSRR寄存器对应的低16位，否则，将该GPIO_Pin左移16位写入BSRR寄存器对应的高16位中。

5.4.1 GPIO的HAL库接口函数

置位/复位寄存器GPIOx_BSRR

9.2.5 Port bit set/reset register (GPIOx_BSRR) (x=A..G)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BRy:** Port x Reset *bit y (y= 0 .. 15)*

These bits are write-only and can be accessed in Word mode only.
0: No action on the corresponding ODRx bit
1: Reset the corresponding ODRx bit

Note: If both BSx and BRx are set, BSx has priority.

Bits 15:0 **BSy:** Port x *Set bit y (y= 0 .. 15)*

These bits are write-only and can be accessed in Word mode only.
0: No action on the corresponding ODRx bit
1: Set the corresponding ODRx bit

- 对GPIOx_BSRR的低16位的某位置“1”，则对应的I/O端口引脚输出高电平“1”；
- 对GPIOx_BSRR的低16位的某位置“0”，则对应的I/O端口引脚状态不变；
- 对GPIOx_BSRR的高16位的某位置“1”，则对应的I/O端口引脚输出低电平“0”；
- 对GPIOx_BSRR的高16位的某位置“0”，则对应的I/O端口引脚状态不变；

GPIOx_BSRR寄存器的高16位称为清除寄存器，低16位称为置位寄存器。

5.4.1 GPIO的HAL库接口函数

源码解析：

读出输出数据寄存器ODR的值与指定引脚进行按位与操作，结果若为真，表明原来为高电平，则写入BRR寄存器(BRR寄存器写入“1”有效，写入“0”不影响ODR的状态）；若结果为假，表明原来为低电平，则控制BSRR寄存器低16位部分做置位（置“1”）操作。

函数源码

```
void HAL_GPIO_TogglePin(GPIO_TypeDef
*GPIOx, uint16_t GPIO_Pin)
{
    if ((GPIOx->ODR & GPIO_Pin) != 0x00u)
    {
        GPIOx->BRR = (uint32_t)GPIO_Pin;
    }
    else
    {
        GPIOx->BSRR = (uint32_t)GPIO_Pin;
    }
}
```

5.4.1 GPIO的HAL库接口函数

数据输出寄存器GPIOx_ODR

9.2.4 Port output data register (GPIOx_ODR) (x=A..G)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODRy:** Port output data (y= 0 .. 15)

These bits can be read and written by software and can be accessed in Word mode only.

Note: For atomic bit set/reset, the ODR bits can be individually set and cleared by writing to the GPIOx_BSRR register (x = A .. G).

GPIOx_ODR数据输出寄存器的高16位为保留位，没有用到；低16位对应I/O端口相应的引脚，可进行置位或复位操作。

5.4.1 GPIO的HAL库接口函数

端口位清除寄存器GPIOx_BRR

9.2.6 Port bit reset register (GPIOx_BRR) (x=A..G)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 Reserved

Bits 15:0 **BRy:** Port x Reset bit y (y= 0 .. 15)
These bits are write-only and can be accessed in Word mode only.
0: No action on the corresponding ODRx bit
1: Reset the corresponding ODRx bit

端口位清除寄存器GPIOx_BRR可以对端口数据输出寄存器GPIOx_ODR中的每位进行复位操作，即GPIOx_BRR只能改变引脚状态为低电平。

◆对GPIOx_BRR的低16位的某位置“1”，则对应的I/O端口引脚置输出低电平“0”；

◆对GPIOx_BRR的低16位的某位置“0”，则对应的I/O端口引脚状态不变。

5.4.1 GPIO的HAL库接口函数

举例

将PB5置为低电平，即将GPIOB_ODR寄存器的第5位ODR5置“0”(注：寄存器的位数为0~15），则可以有三种方式实现:

方法一

操作GPIOx_ODR寄存器
GPIOB->ODR = 0xffef(1111 1111 1101 1111)；

位	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0020	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1

方法二

操作GPIOx_BSRR寄存器
GPIOB->BSRR = 0x00200000(0000 0000 0010 0000 0000 0000 0000 0000)；

方法三

操作GPIOx_BRR寄存器
GPIOB->BRR = 0x0020(0000 0000 0010 0000)；

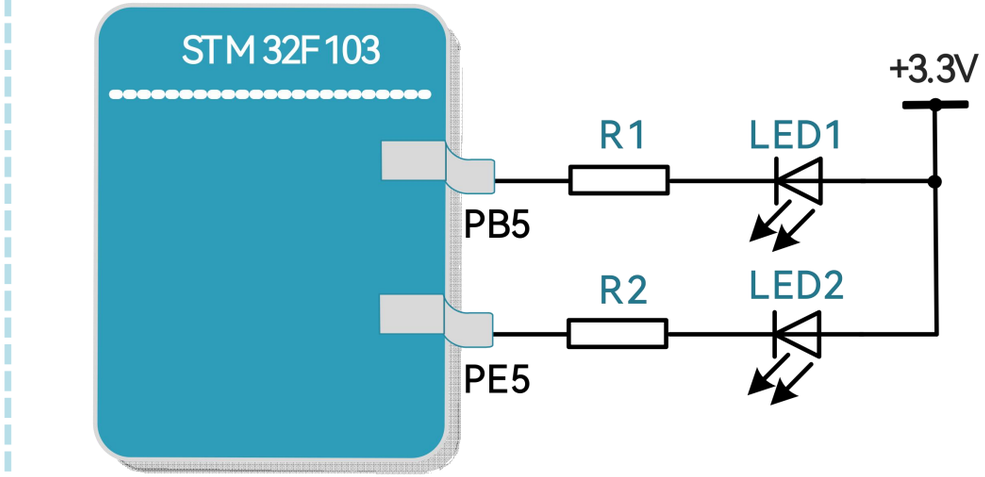
位	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0020	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0

5.4.2 GPIO的HAL库应用实例

功能

采用基于HAL库设计方式，利用两个GPIO引脚输出高低电平控制发光二极管，并按一定时间间隔改变IO口电平，实现灯光闪烁效果。

硬件设计

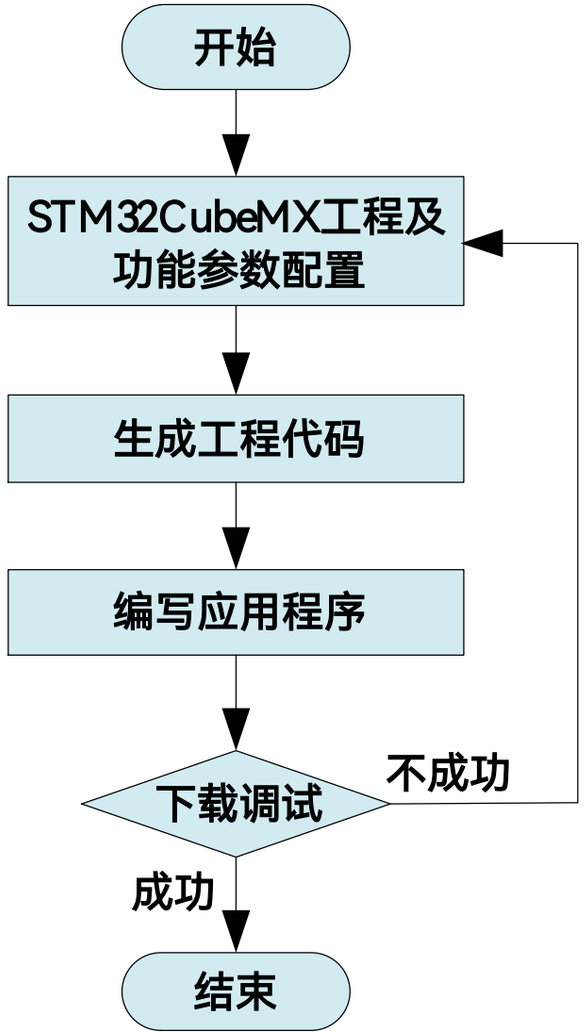


当引脚输出为低电平时，LED灯亮，当引脚输出为高电平时，LED灯灭。

5.4.2 GPIO的HAL库应用实例

基于HAL库的软件设计流程如图所示：

- 建立STM32CubeMX工程；
- 进行功能参数配置；
- 生成工程代码；
- 修改代码完成应用程序设计；
- 下载到开发板测试。



设计步骤——新建STM32CubeMX工程，选择设计采用的MCU

在D盘或其他盘符目录下新建一个文件夹，用来存放后面建立的STM32CubeMX工程。

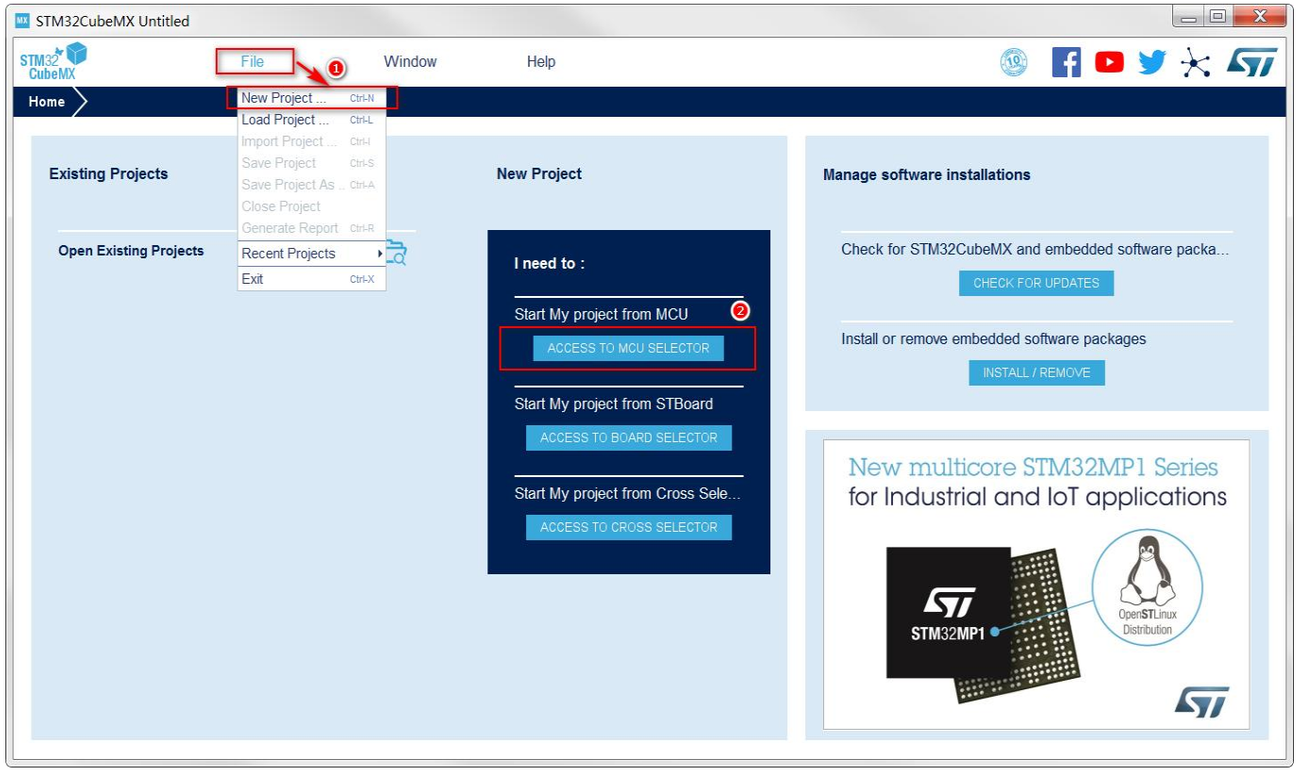


所建工程文件名必须是英文名称，且必须是英文路径。

设计步骤——新建STM32CubeMX工程，选择设计采用的MCU

打开STM32CubeMX软件

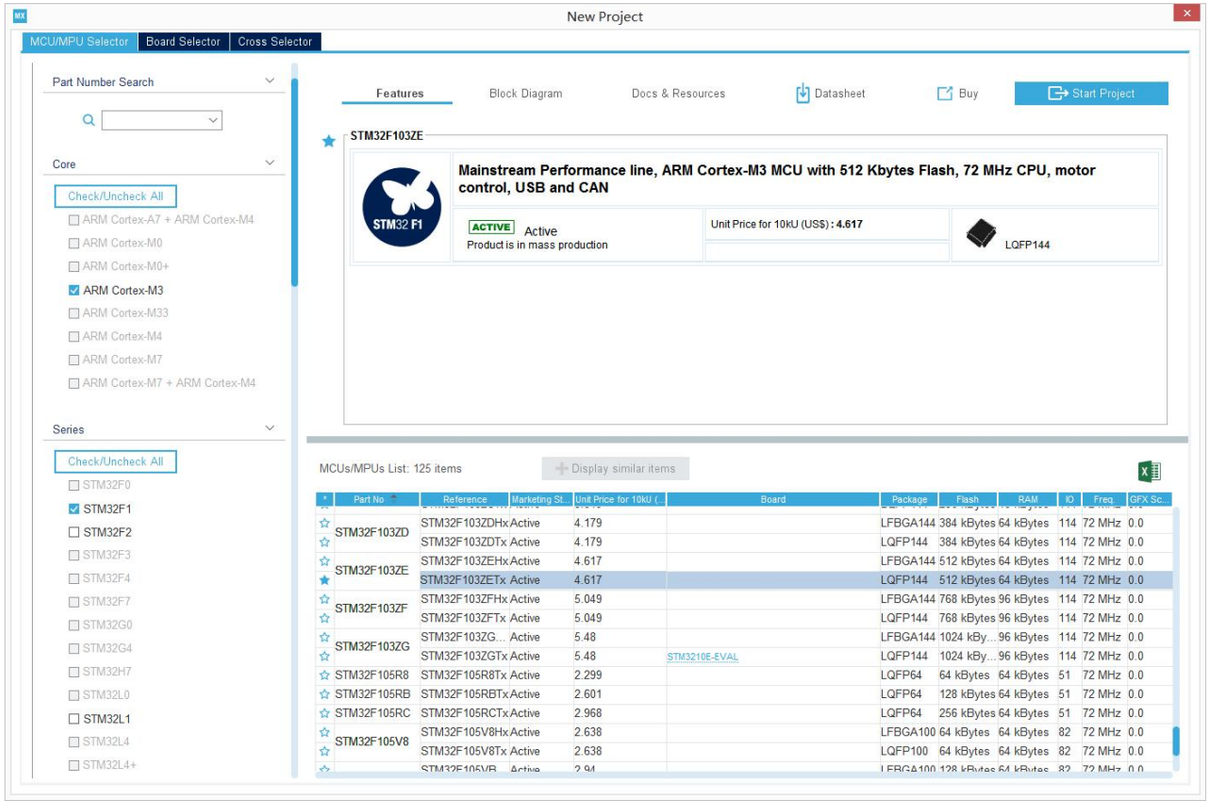




在主界面中通过单击“New Project”下“ACCESS TO MCU SELECTOR”按钮或通过菜单栏中的“File”→“New Project”新建一个工程

设计步骤——新建STM32CubeMX工程，选择设计采用的MCU

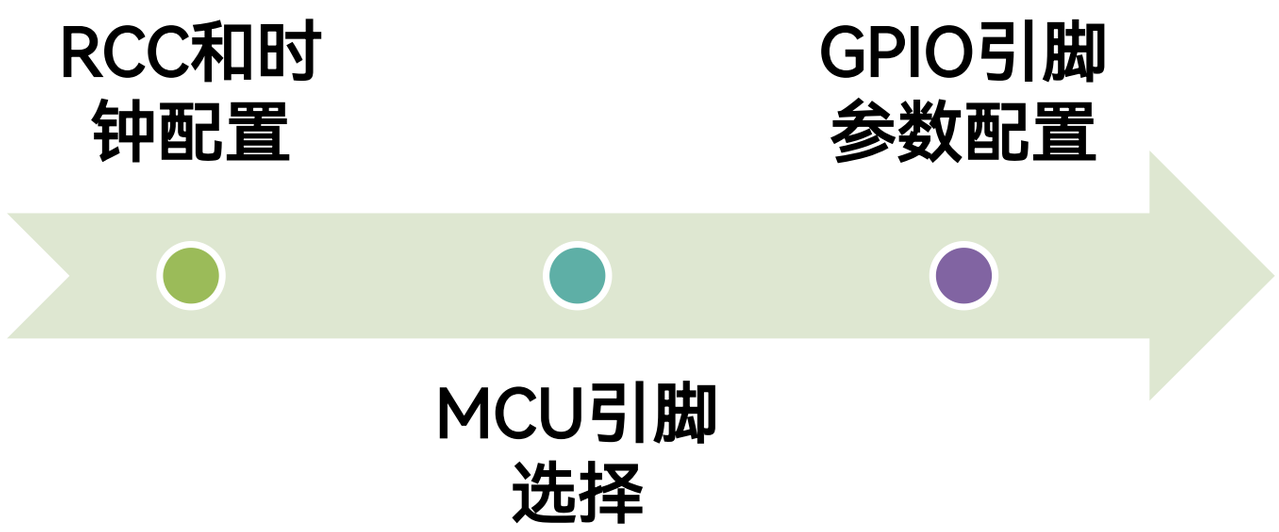
- 选择所用MCU对应的内核，选择“ARM Cortex-M3”；
- 在“Series”选项中选择对应的系列，如“SMT32F1”；
- 在右侧“MCUs/MPUs List”中找到该系列下所使用的微控制器芯片型号
- 选中并单击对应芯片型号，在“MCUs/MPUs List”上方会显示该芯片对应的基本信息。



也可以通过在搜索栏内直接输入芯片型号采用搜索的方式来快速完成。

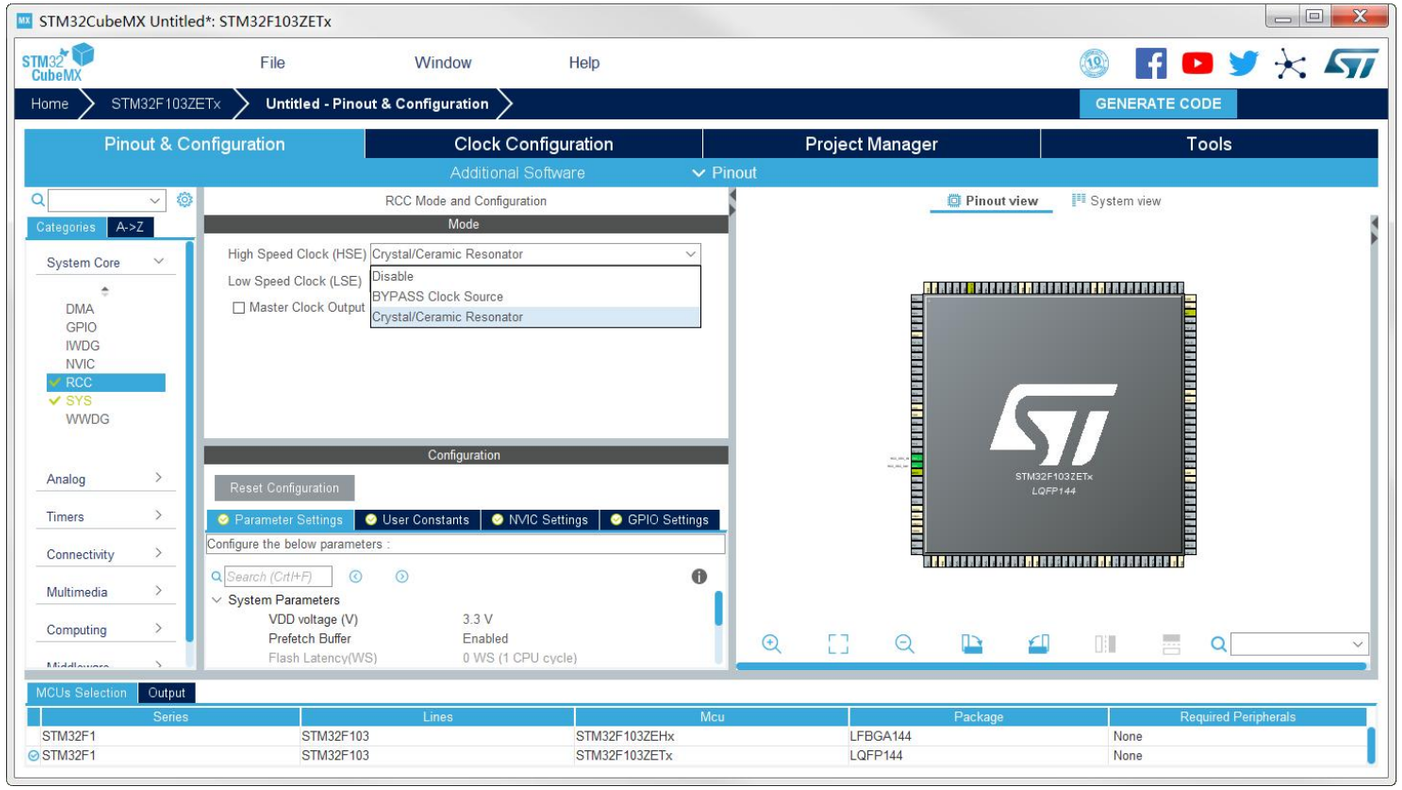
设计步骤——STM32CubeMX功能参数配置

在New Project窗口完成MCU的相关设置后，双击所选择的具体芯片，进入STM32CubeMX的主界面，完成以下过程：



设计步骤——STM32CubeMX功能参数配置（RCC配置）

在“Categories”栏目中的“System Core”中，找到“RCC”选项

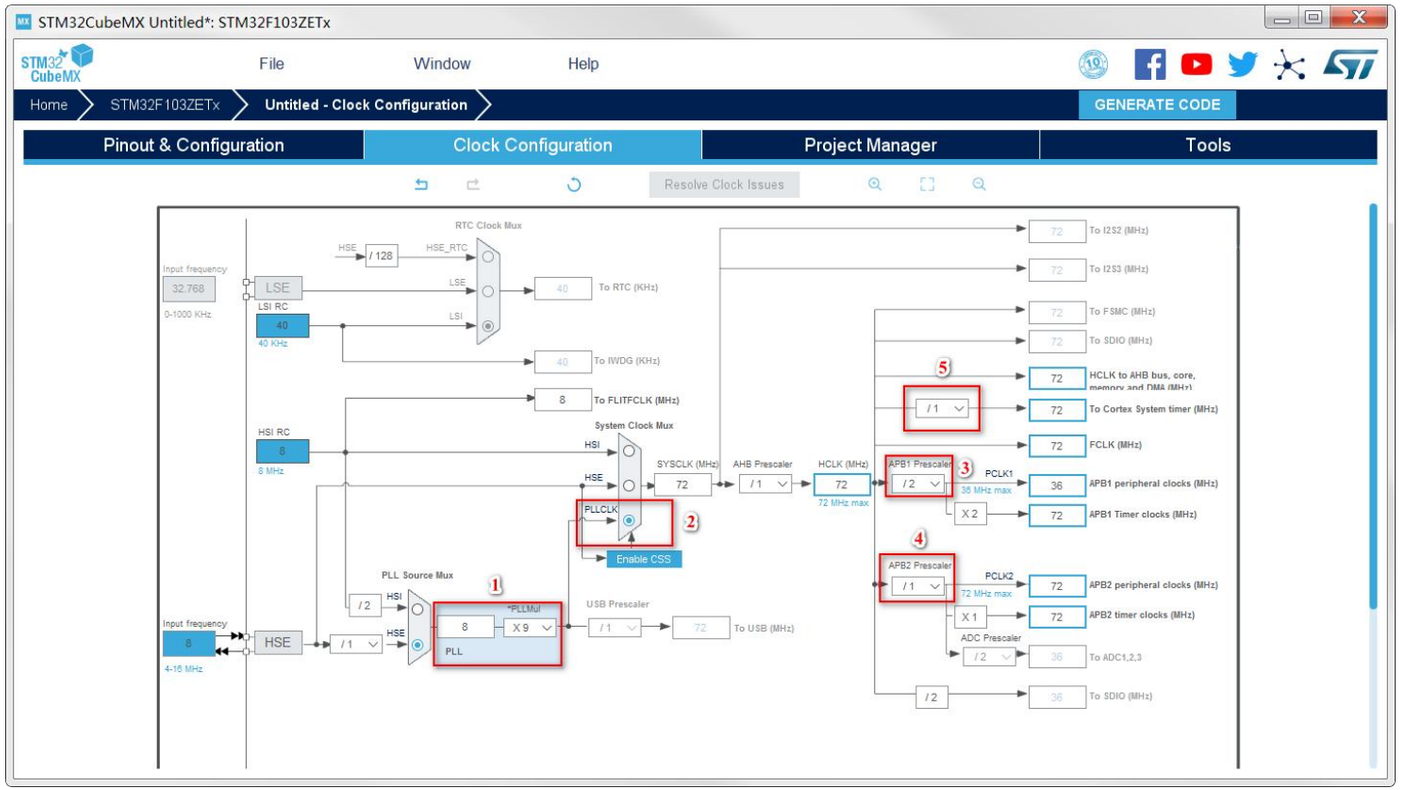


时钟信号选择HSE作为系统的外部时钟源，HSE选择“Crystal/Ceramic Resonator”（晶振/陶瓷谐振器）

LSE选择“Disable”

设计步骤——STM32CubeMX功能参数配置(时钟配置)

点开“Clock Configuration”选项栏，进行系统时钟配置



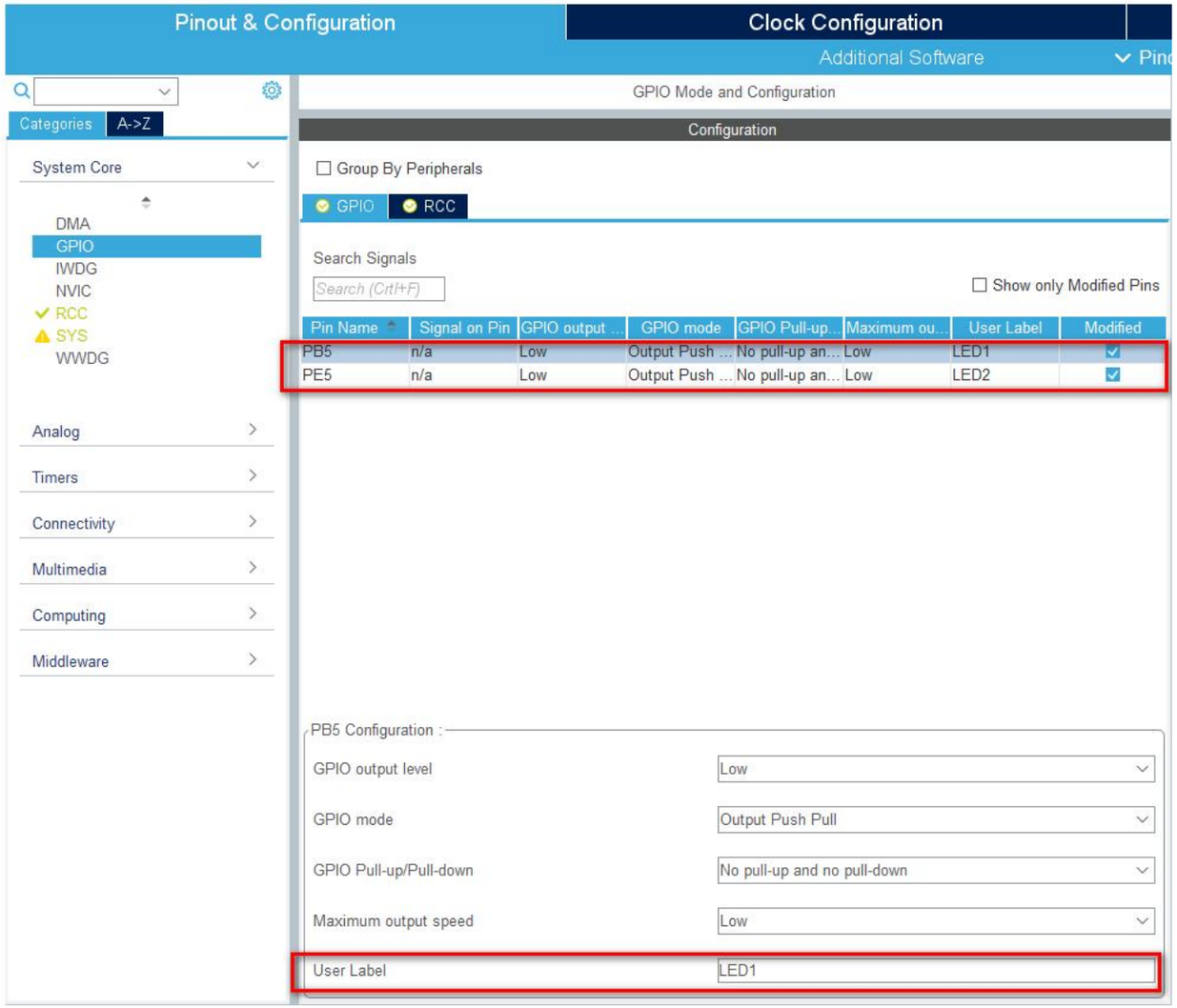
- 采用HSE外部晶振，频率为8MHz，通过PLL的9倍频，使得系统时钟SYSCLK为72MHz；
- APB2时钟与HCLK相同，所以不需要分频；
- HCLK经2分频得到APB1的时钟频率为PCLK1=36MHz，

设计步骤——STM32CubeMX功能参数配置(MCU引脚选择)

在STM32CubeMX的主界面的左侧“Categories”栏目中的“System Core”中，找到“GPIO”选项，在STM32CubeMX的主界面的右侧“Pinout view”芯片引脚图中选中PE5、PB5引脚，设置为GPIO_Output选项。



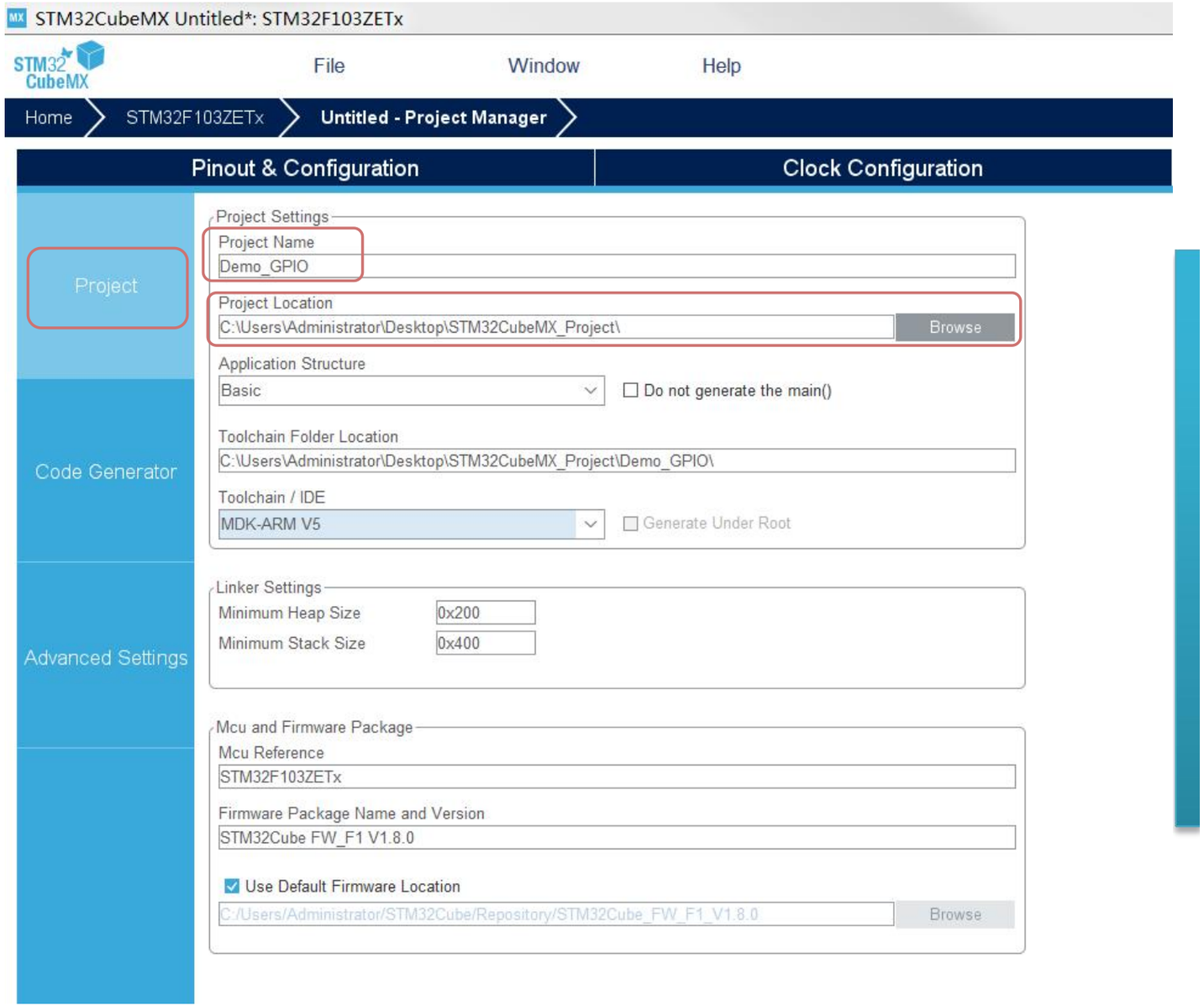
设计步骤——STM32CubeMX功能参数配置（GPIO引脚参数配置）



选择好引脚后，在左侧“Configuration”栏目下显示所选择的引脚，单击，对应显示该引脚的参数设置表，设置相应参数，如输出电平、模式、用户标签等。

设计步骤——生成工程代码

点击STM32CubeMX的主界面中的“Project Manager”菜单，在弹出的页面中单击“Project”项，输入项目名称“Demo_GPIO”，选择存放路径。

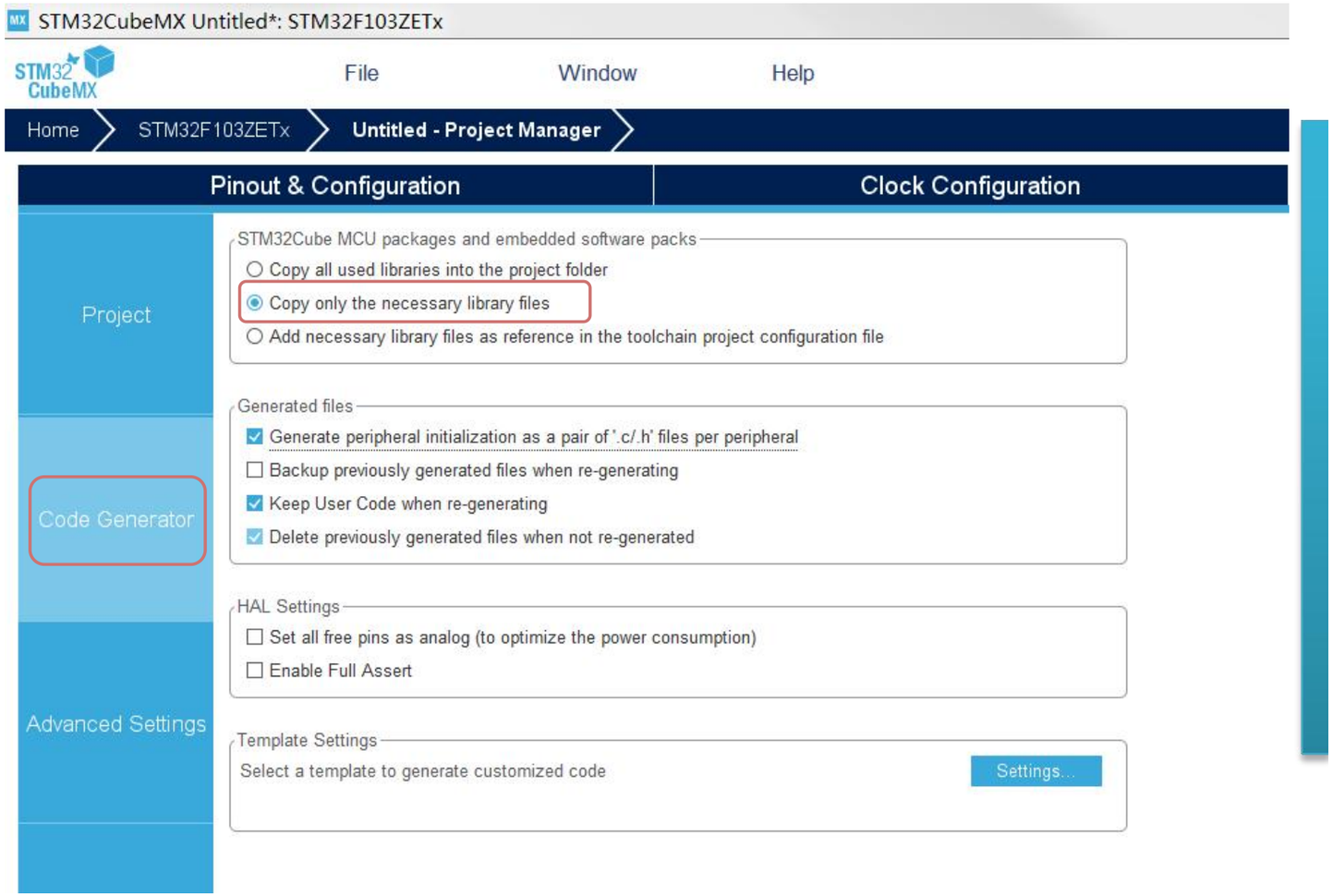


说明

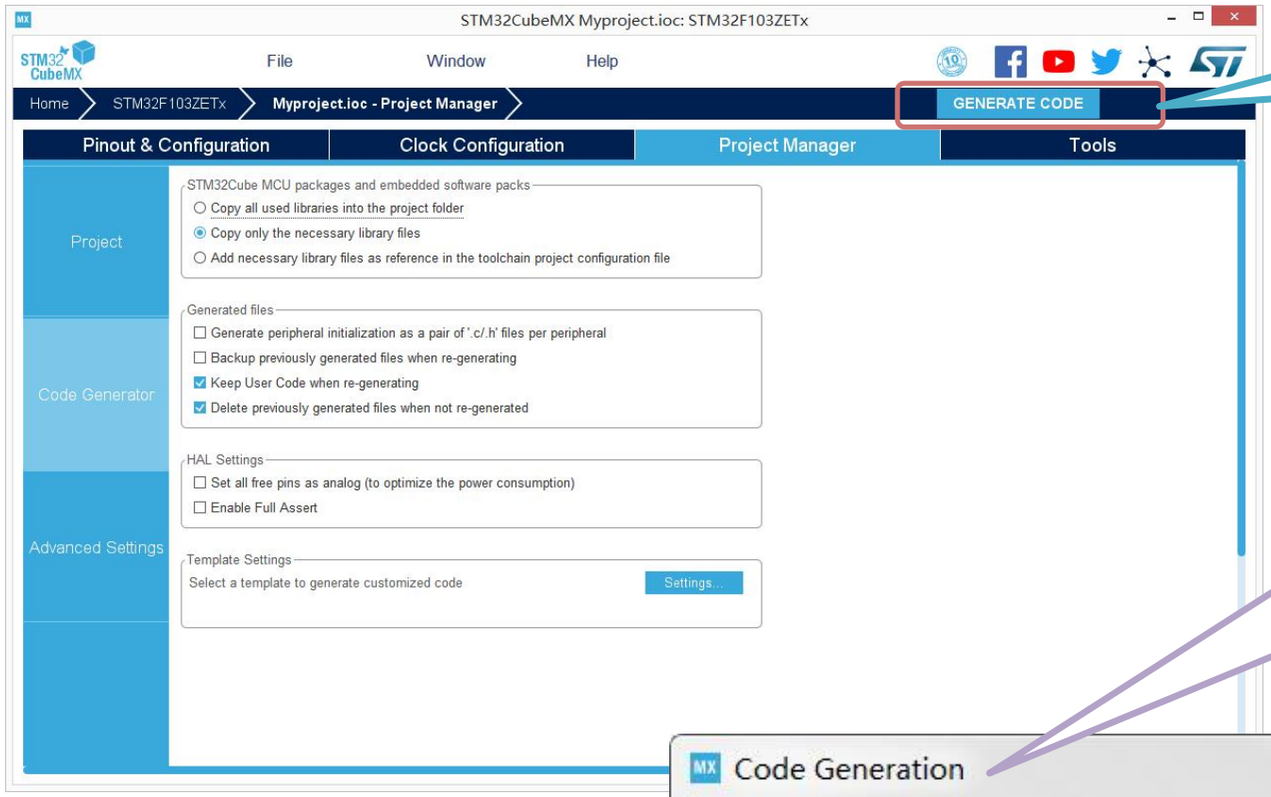
用户可以自己定义，项目名称一般应反映项目内容，便于管理

设计步骤——生成工程代码

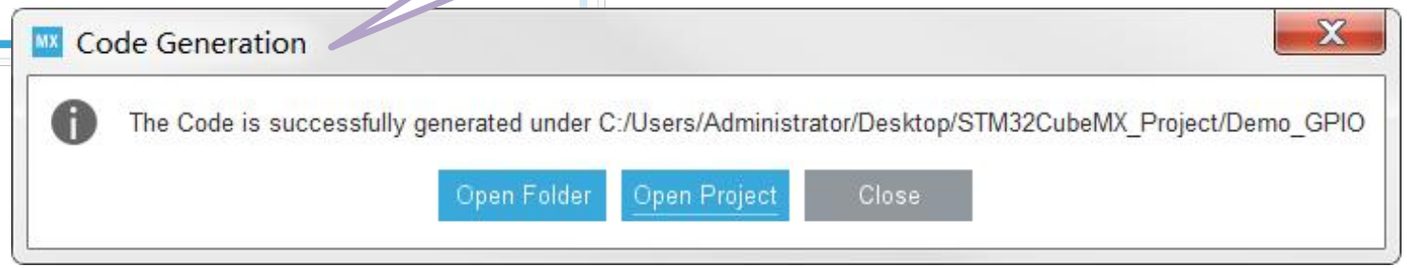
- 点击“Code Generator”选项
- 在“STM32Cube MCU packages and embedded software packs”栏内选择第二个单选项“Copy only the necessary library files”，仅拷贝必须的库文件；
- 在“Generated files”栏内复选第1、3、4选项



设计步骤——生成工程代码



单击右上角“GENERATE CODE”按钮，即可生成对应工程代码。



成功后弹出Code Generation窗口，提示代码生成成功，用户可以根据下一步需要选择打开文件夹、打开项目或关闭。

设计步骤——编写应用程序

本实例的用户应用程序代码写在/* USER CODE BEGIN 3 */ 和 /* USER CODE END 3 */ 之间：

```
while (1)
{
  /* USER CODE END WHILE */
  /* USER CODE BEGIN 3 */

HAL_GPIO_TogglePin(LED1_GPIO_Port,LED1_Pin);
          //LED1---PB5状态翻转
    HAL_Delay(100); //延时100毫秒

HAL_GPIO_TogglePin(LED2_GPIO_Port,LED2_Pin);
          //LED2---PE5状态翻转
    HAL_Delay(100); //延时100毫秒
```

使用STM32CubeMX生成的工程，其用户功能代码即应用程序代码的编写有位置规范要求。

说明

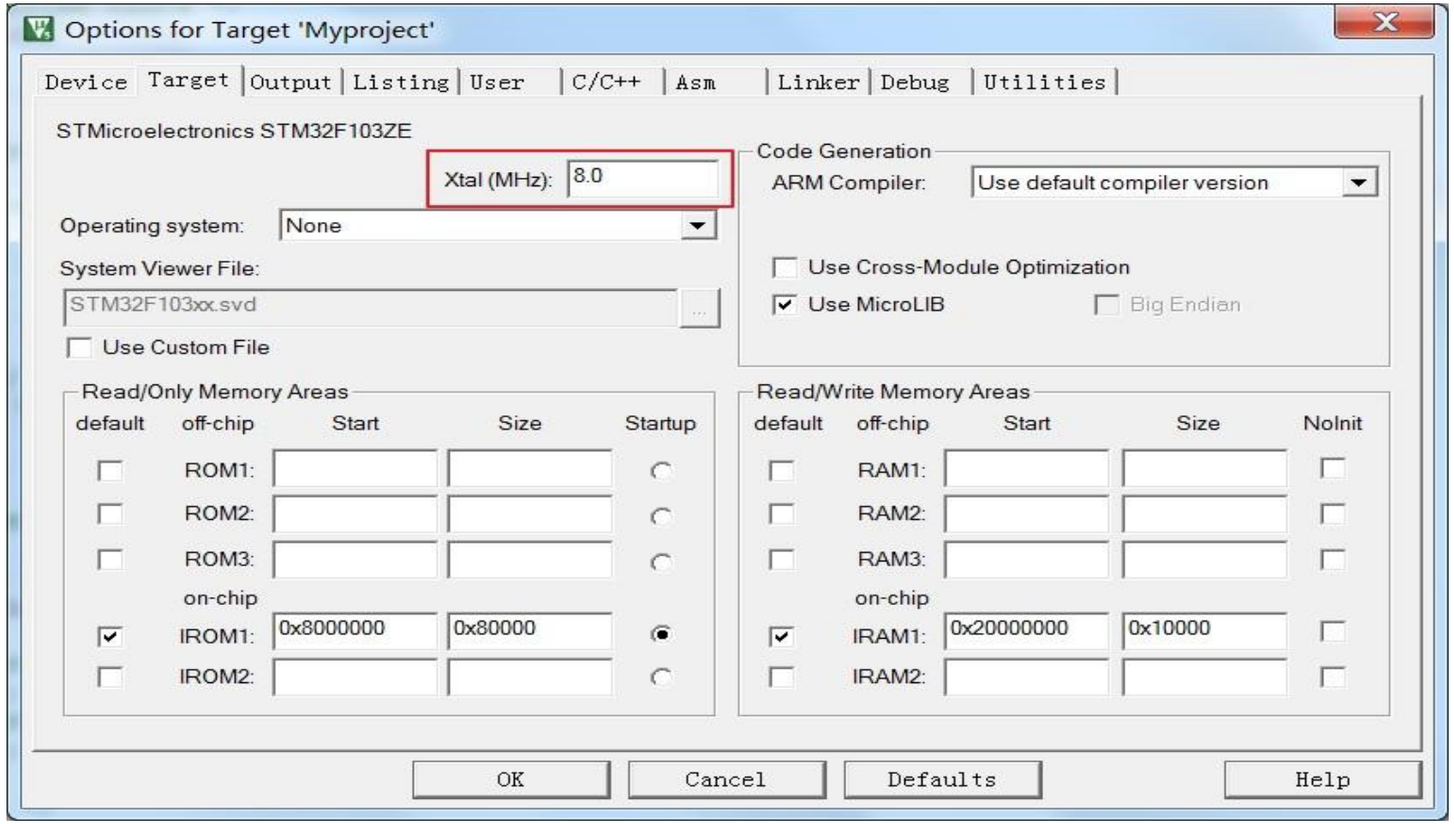
用户自己编写的应用程序需写在/* USER CODE BEGIN x */ 和 /* USER CODE END x */ 之间。

注意

写在其他地方，在使用STM32CubeMX重新配置和生成工程时，会删除该代码。

设计步骤——下载调试验证

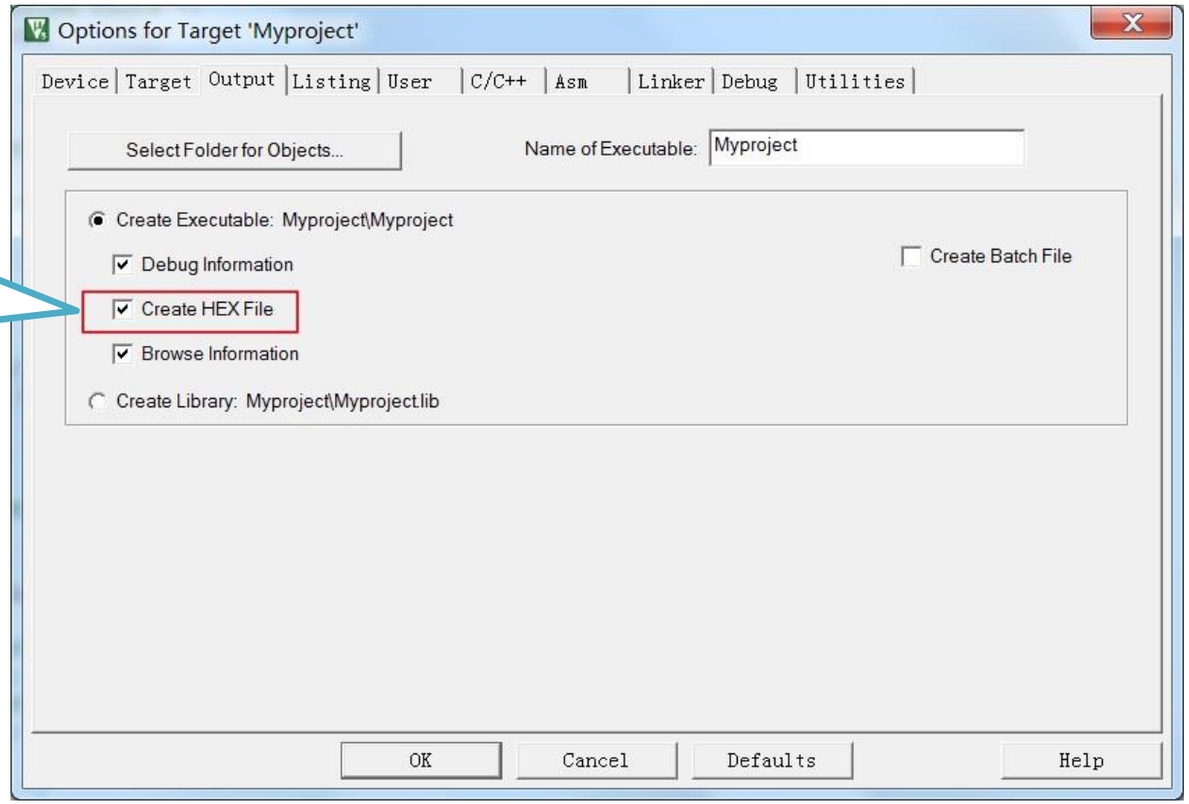
配置Keil5相关工程，重新编译。



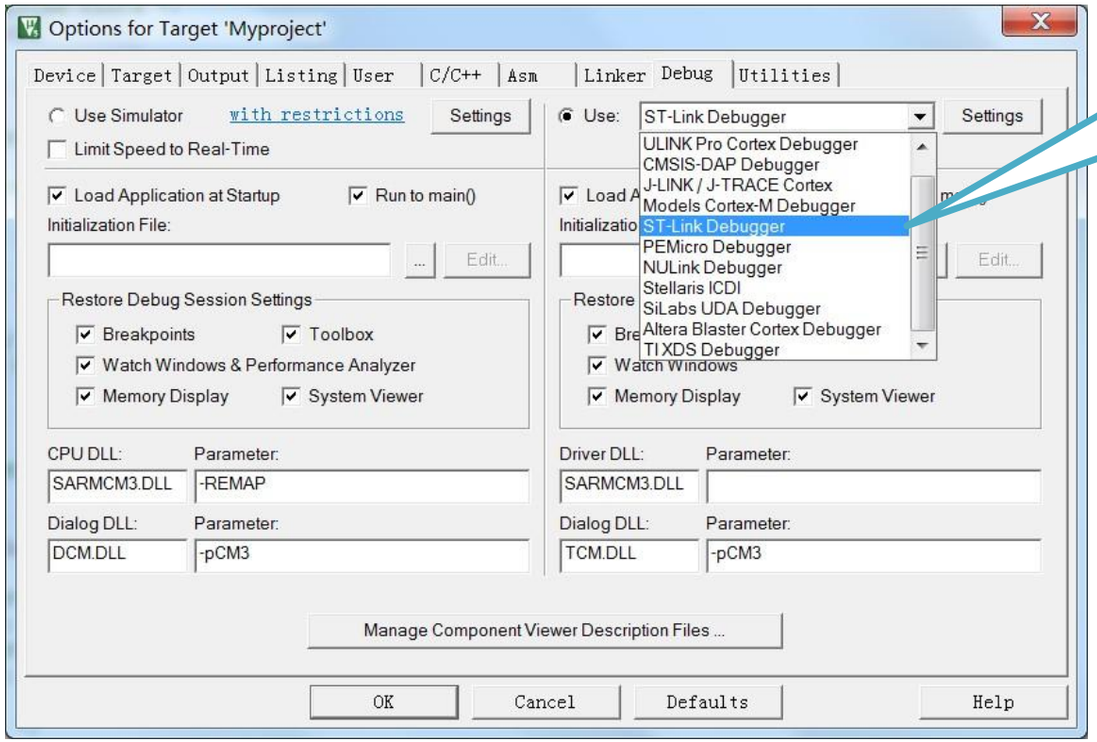
单击keil uVision5工具栏的魔法棒“Options for Target”按钮，打开配置窗口页面，在“Target”选项卡中将“Xtal(MHz)”改为8.0，采用8MHz外部晶振

设计步骤——下载调试验证

在Output选项卡中，勾选Create HEX File选项，在工程重新编译后，会生成相应的HEX文件。



设计步骤——下载调试验证



在“Debug”选项卡中，根据所使用的开发板下载调试工具，选择相应的选项，这里选择ST-Link调试下载器

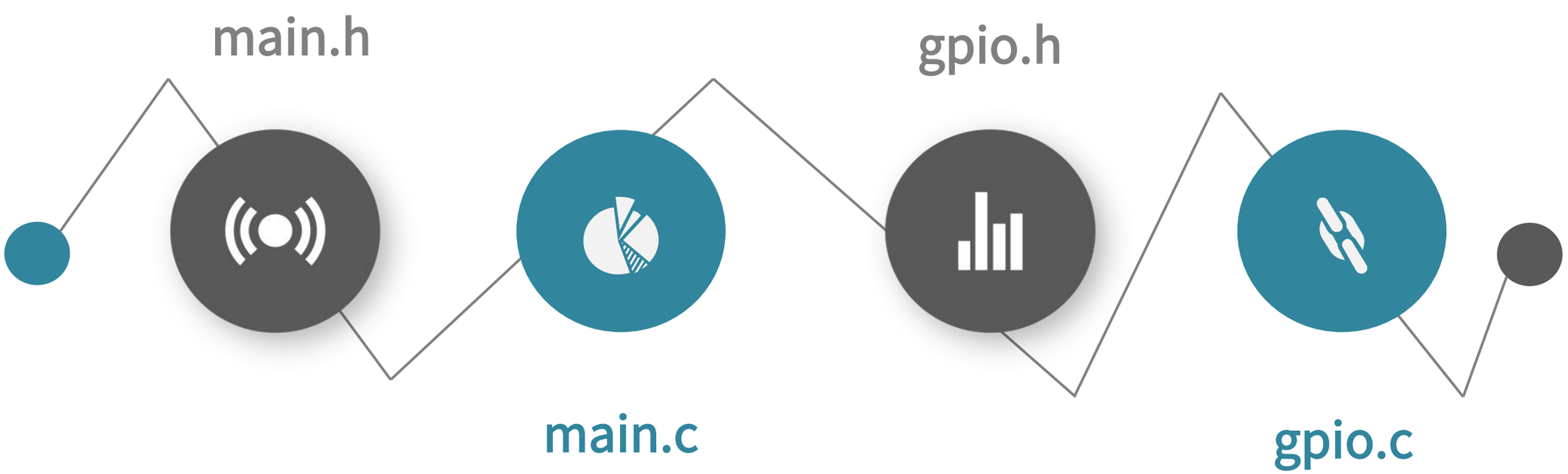
单击“Build(F7)”编译按钮，进行重新编译。

将开发板通过USB线与PC机进行连接

通过工具栏里的“Download”按钮将编译通过的程序下载到开发板上

通过观察开发板上相应LED灯的闪烁情况来验证程序的正确性。

GPIO工程源码解析



针对GPIO-LED闪烁工程相关源码共四个文件

GPIO工程源码解析

main.c

```
        /* Includes 头文件*/
#include "main.h"
#include "gpio.h"

      /* 函数声明 */
void SystemClock_Config(void); //设置系统时钟

int main(void)
{
          /* 将所有的外设复位，
              并初始化Flash和系统滴答时钟Systick */
  HAL_Init();
        /*配置系统时钟*/
  SystemClock_Config();
         /***GPIO初始化，函数实现定义在gpio.c文件中
           *此代码对应于STM32CubeMX软件中的Pinout
&
           Configuration设置的引脚参数
         */
  MX_GPIO_Init();
while (1)
  {
        //以下为用户编写的应用程序代码

          /*调用HAL_GPIO_TogglePin函数
             使LED1---PB5状态翻转 */

HAL_GPIO_TogglePin(LED1_GPIO_Port,LED1_Pi
n);
HAL_Delay(100);
          //调用HAL库函数HAL_Delay延时100毫秒
          /*调用HAL_GPIO_TogglePin函数
             使LED2---PE5状态翻转 */

HAL_GPIO_TogglePin(LED2_GPIO_Port,LED2_Pi
n);
    HAL_Delay(100);
           //调用HAL库函数HAL_Delay延时100毫秒
  }
}
```

GPIO工程源码解析

main.c

```
        /***系统时钟配置，对应于STM32CubeMX软件中的系统时钟操作*/
void SystemClock_Config(void)
{
          /*OSC外部晶振初始化结构体变量，用于打开HSE，设置PLL等 */
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
        /*SYSCLK初始化结构体变量，用于选择sysclk的时钟源、设置AHB预分频系数*/
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
        /* 给RCC_OscInitStruct的成员赋值，用于选择系统时钟源，设置PLL、PLLMul的值等
*/
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;//HSE为外部晶振8MHz
  RCC_OscInitStruct.HSEState = RCC_HSE_ON; //打开HSE
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;//设置HSE预分频系数为1
  RCC_OscInitStruct.HSIState = RCC_HSI_ON; //打开HSI(高速内部时钟，HSI=8MHz)
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;//打开PLL
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;//PLL的输入时钟选择为HSE
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;//设置PLLMul为9，倍频到72MHz
```

GPIO工程源码解析

main.c

```
        /***调用HAL_RCC_OscConfig()函数对系统时钟进行初始化配置， 通过if语句判断是否成功，
             如果初始化配置不成功，则由Error_Handler()进行处理*/
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {      Error_Handler();//出错处理函数     }
       /*给RCC_ClkInitStruct的成员赋值，用于选择SYSCLK的时钟源， 设置AHB、APB1、APB2的时钟，
        此代码对应于STM32CubeMX软件中的Clock Configuration配置的系统时钟操作 */
 RCC_ClkInitStruct.ClockType =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK|RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
          /*将 SYSCLK的时钟源选为PLLCLK */
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
          /* 将AHB Prescaler的分频值设为 1，AHB时钟频率=72MHz*/
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
          /* 将APB1 Prescaler的分频值设为2，APB2时钟频率=PCLK1 = 36 MHz*/
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
          /*将APB2 Prescaler的分频值设为1，APB2时钟频率=PCLK2 = 72MHz */
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
          /*判断SYSCLK时钟初始化配置是否成功， 如果初始化配置不成功，则由Error_Handler()进行处理*/
if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {    Error_Handler();      }
}
```

GPIO工程源码解析

main.c

```
/* 错误处理函数 */
void Error_Handler(void)
{
        //错误处理代码由用户编写
}


/* 断言函数，如果使用断言的话，需定义宏USE_FULL_ASSERT，这里没有使用断言*/
#ifdef  USE_FULL_ASSERT
void assert_failed(uint8_t *file, uint32_t line)
{
    //断言失败函数，具体处理代码由用户编写
}
#endif /* USE_FULL_ASSERT */
```

GPIO工程源码解析

gpio.c

```
/* Includes 头文件*/
#include "gpio.h"

/***GPIO初始化，方式和标准外设库的GPIO_Init类似
*此代码对应于STM32CubeMX软件中的Pinout & Configuration设置的引脚参数*/

void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

      /* 使能GPIOB、GPIOE端口时钟 */
  __HAL_RCC_GPIOE_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

     /* 配置GPIO初始引脚LED2-PE5为低电平 */
  HAL_GPIO_WritePin(LED2_GPIO_Port, LED2_Pin,
GPIO_PIN_RESET);

    /* 配置GPIO初始引脚LED1-PB5为低电平 */
  HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin,
GPIO_PIN_RESET);

        /* LED2-PE5引脚参数配置 */
  GPIO_InitStruct.Pin = LED2_Pin; //I/O引脚设置为LED2-
PE5
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
      //引脚模式设置为推挽输出
  GPIO_InitStruct.Pull = GPIO_NOPULL; //无上拉
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
       //输出速度为低速
  HAL_GPIO_Init(LED2_GPIO_Port, &GPIO_InitStruct);
       //调用初始化函数初始化PE5

       /* LED1-PB5引脚参数配置 */
  GPIO_InitStruct.Pin = LED1_Pin; //I/O引脚设置为LED1-
PB5
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
       //引脚模式设置为推挽输出
  GPIO_InitStruct.Pull = GPIO_NOPULL; //无上拉
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
       //输出速度为低速
  HAL_GPIO_Init(LED1_GPIO_Port, &GPIO_InitStruct);
       //调用初始化函数初始化PB5
}
```

GPIO工程源码解析

main.h

```
    /*条件编译，防止头文件被重复包含*/
#ifndef __MAIN_H
#define __MAIN_H

#ifdef __cplusplus
extern "C" {
#endif

    /* Includes 头文件,stm32f1xx_hal.h这个头文件很重要*/
#include "stm32f1xx_hal.h"
```

```
    /* 函数声明 */
void Error_Handler(void);
    /* 端口引脚的宏定义，方便修改 */
#define LED2_Pin GPIO_PIN_5
#define LED2_GPIO_Port GPIOE
#define LED1_Pin GPIO_PIN_5
#define LED1_GPIO_Port GPIOB

#ifdef __cplusplus
}
#endif

#endif /* __MAIN_H */
```

GPIO工程源码解析

源码gpio.h

```
        /* 条件编译，防止头文件被重复包含*/
#ifndef __gpio_H
#define __gpio_H
#ifdef __cplusplus
extern "C" {
#endif
    /* Includes 头文件*/
#include "main.h"
    /* 函数声明 */
void MX_GPIO_Init(void);
#ifdef __cplusplus
}
#endif
#endif /*__ pinoutConfig_H */
```

5.4.3 基于HAL库开发的一般流程

类似于基于标准库的开发，基于HAL库的开发主要有以下流程：

开启系统时钟	配置系统时钟树	外设参数配置	生成工程代码	应用程序编写
• 在STM32CubeMX主界面中的“Pinout & Configuration”选项页将RCC设置系统外部时钟源为HSE，HSE选择“Crystal/Ceramic Resonator”（晶振/陶瓷谐振器）	• 将外部的8MHz晶振经PLL倍频9倍得到72MHz的系统时钟，然后设置相应外设时钟，如APB1为系统时钟的2分频为36MHz，APB2设置为与系统时钟相同	• 在STM32CubeMX主界面中的“Pinout & Configuration”选项页对片上外设进行功能配置，如GPIO、ADC、定时器、NVIC等	• 设置好工程所要存放的位置及所使用的集成开发环境，就可以生成工程代码	• STM32CubeMX配置完成后，生成工程代码，就可以在Keil软件中编写应用程序，以实现相应功能。

5.5 编程思想之模块化编程

- 拓展1：通过已有的库函数实现更为复杂的功能函数，通过对函数的进一步封装，来简化函数调用。

例：通过HAL库的WritePin函数和ReadPin函数，实现TogglePin（I/O状态翻转）功能函数

```
void BSP_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
{
    HAL_GPIO_WritePin(GPIOx,GPIO_Pin,( GPIO_PinState)((1-
GPIO_ReadPin(GPIOx,GPIO_Pin))));
    HAL_Delay(h);
}
```

5.5 编程思想之模块化编程

在主函数中通过右侧代码，实现函数调用。

```
int main(void)
{
    ……
    MX_GPIO_Init();  // I/O引脚硬件驱动配置初始化
    while (1)
     {
          //将PE5引脚状态每隔100ms进行翻转

BSP_GPIO_TogglePin(GPIOE,GPIO_PIN_5,100);
     }
}
```

5.5 编程思想之模块化编程

◆ 拓展2：模块化编程——BSP板级驱动函数设计

BSP：Board Support Packages，板级支持包。

BSP就是一个介于操作系统和底层硬件之间的软件层次，包括了系统中大部分与硬件相关的软件模块，在功能上包含两部分：系统初始化及与硬件相关的设备驱动。

应用程序层
通过调用BSP和HAL，实现系统功能
BSP（功能模块层）
· 通过调用HAL，实现项目中所用外设功能模块，如OLED屏、GSM模块等； · 隐藏具体的外设模块操作细节； · 为上层提供调用接口。
HAL（硬件抽象层）
· 对片内硬件资源（如GPIO、中断、定时器、ADC等）的通用配置； · 隐藏具体的寄存器操作细节； · 为上层提供调用接口。

BSP板级驱动函数是在HAL库的基础上进行的再次抽象和封装。

5.5 编程思想之模块化编程

◆ 拓展2：模块化编程——BSP板级驱动函数设计

模块化的板级驱动函数编程设计以.c源文件和.h头文件的形式实现，.h头文件则用于声明.c源文件中的功能函数、用于隔离和封装底层具体硬件的宏定义及枚举类型、结构体类型等数据类型的定义。

```
#ifndef __LED_H__
#define __LED_H__
#include “stm32f1xx_hal.h”
//采用宏定义，将指示灯LED0连接的引脚和端口分别定义一个别名
#define led0_port GPIOC
#define led0_pin GPIO_PIN_3
//采用宏定义，将指示灯LED1连接的引脚和端口分别定义一个别名
#define led1_port GPIOE
#define led1_pin GPIO_PIN_5
//采用宏定义，将指示灯LED2连接的引脚和端口分别定义一个别名
#define led2_port GPIOB
#define led2_pin GPIO_PIN_5
//声明函数
void BSP_LED_On(uint16_t number);
void BSP_LED_Off(uint16_t number);
#endif
```

5.5 编程思想之模块化编程

.c源文件是具体业务逻辑实现，即功能函数实现，包含一个或多个具体的功能函数。

在led.c文件中设计一个BSP_LED_On (uint16_t led_unmber)和void BSP_LED_Off (uint16_t led_unmber)函数，其中形参led_unmber为目标板上具体对应某个设备指示灯灯编号，取值为0、1或2。

◆ 拓展2：模块化编程——BSP板级驱动函数设计

```
#include "stm32f1xx_hal.h"
#include "led.h"
//指示灯亮函数，GPIO引脚为低电平时有效
void BSP_LED_On (uint16_t led_unmber)
{
    switch(led_unmber)
    {   case 0:
        HAL_GPIO_WritePin(led0_port,
led0_pin,GPIO_PIN_RESET);
        break;
        case 1:
        HAL_GPIO_WritePin(led1_port,
led1_pin,GPIO_PIN_RESET);
        break;
        case 2:
        HAL_GPIO_WritePin(led2_port,
led2_pin,GPIO_PIN_RESET);
        break;
        default:
         break;
```

5.5 编程思想之模块化编程

```
//指示灯灭函数，GPIO引脚为高电平时有效
void BSP_LED_Off (uint16_t led_unmber)
{
    switch(led_unmber)
    { case 0:
       HAL_GPIO_WritePin(led0_port,
led0_pin,GPIO_PIN_SET);
       break;
       case 1:
       HAL_GPIO_WritePin(led1_port,
led1_pin,GPIO_PIN_SET);
       break;
       case 2:
       HAL_GPIO_WritePin(led2_port,
led2_pin,GPIO_PIN_SET);
       break;
       default:
       break;
    }
```

◆ 拓展2：模块化编程——BSP板级驱动函数设计

.c源文件是具体业务逻辑实现，即功能函数实现，包含一个或多个具体的功能函数。

在led.c文件中设计一个BSP_LED_On (uint16_t led_unmber)和void BSP_LED_Off (uint16_t led_unmber)函数，其中形参led_unmber为目标板上具体对应某个设备指示灯灯编号，取值为0、1或2。

本章小结

5.1 GPIO概述

5.2 STM32的GPIO工作原理

- GPIO引脚
- GPIO内部结构
- GPIO工作模式

5.3 GPIO模块的标准外设库接口函数及应用

- 接口函数
- 应用实例

5.4 GPIO模块的HAL库接口函数及应用

- 接口函数
- 应用实例

5.5 编程思想之模块化编程



The End!

